

Partners in Crime: Attempting to Infer Combinatorial Regulation from Bulk Transcriptomics

Koen van den Berk

Supervised by:
Christopher Hojny
Federica Eduati
Bi-rong Wang

September 2026

Contents

1	Introduction	1
1.1	Data	3
2	Boolean model	5
2.1	Model description	7
2.2	Pricing problem	10
2.3	Implementation	11
2.4	Methodology	14
2.5	Results	16
3	Threshold model	19
3.1	Model description	19
3.2	Implementation	21
3.3	Methodology	21
3.4	Results	22
3.5	Greedy algorithm	28
4	Ramp loss model	31
4.1	Model description	32
4.2	Pricing problem	35
4.2.1	Complexity	36
4.2.2	Mixed Integer Programming formulations	38
4.2.3	Cutting planes	40
4.2.4	Separation	41
4.3	Examples	44
4.4	Non-additive variant	50
4.5	Implementation	53
4.6	Methodology	54
4.7	Results	54
5	Additional experiments	57
5.1	Tissue status	57
5.2	Gene expression as input	59

5.3 Transcription Factor Exclusion	61
6 Performance on holdout dataset	63
7 Recommendations	65
8 Conclusion	67
Use of Artificial Intelligence	68
A Data	73
B Reduced costs	77
B.1 Boolean model	77
B.2 Additive ramp loss model	78
B.3 Non-additive ramp loss model	78
C Cutting planes	81
C.1 Mixed Vertex Packing Problem	81
C.2 Extended Polymatroid Inequalities	82
D Results of experiments	85
D.1 Boolean model integer gap	85
D.2 Ramp loss model generalization	87
D.3 Gene expression as input	88
D.4 Transcription Factor Exclusion	91

Chapter 1

Introduction

TODO another introductory paragraph before this one. Perhaps an analogy, perhaps something simple to introduce the topic of cancer? Or is the following line good enough?

One of the ways that cancerous tissue tries to evade the immune system is through the PD-L1 protein. The PD-L1 protein serves as a signal to immune cells not to attack [28]. Besides being abused by cancer cells, it is used by healthy cells to prevent an overreaction of the immune system. PD-L1 is encoded by the CD274 gene. For the PD-L1 protein to be synthesized, the gene first needs to be transcribed. To start transcription, certain proteins need to bind to the gene sequence. These proteins are called transcription factors, or TF for short. It takes several TF, bound together in a complex, to start transcription. Biologists are interested in which TF act together to enable transcription. Who are these “partners in crime”, helping PD-L1 be synthesized and ultimately helping cancer go unnoticed? Of course, the TF that enable production of PD-L1 in cancer cells may be the same ones acting in healthy cells. Still, for the sake of investigation, we shall consider these TF “partners in crime” and embark on an investigative journey.

The aim of this research is to identify, from data, candidate groups of TF that may act together to enable PD-L1 production. The candidate groups could then be studied further, for instance in lab experiments. Understanding which groups are enabling the PD-L1 evasion tactic may ultimately help to develop strategies to prevent it. For even moderate number of TF and moderate group sizes, the number of possible groups is large. Consequently, it is attractive or even necessary to study candidate groups systematically and computationally.

We wish to design computational methods capable of identifying these

groups from bulk transcriptomics, as such data is widely available. We wish to use TF activity data for our analysis, as a TF needs to be active to bind to a gene and contribute to transcription. Measuring how active a TF is inside a cell, however, is difficult. Readily available are bulk RNA sequencing data. This measures the average number of RNA transcripts across an entire group of cells in a tissue sample. The RNA transcript count will be simply referred to as “gene expression”. Gene expression is not equivalent to protein activity. If gene expression is high, it does not necessarily mean that the resulting protein is active. Intermediate steps can affect the protein activity, like protein synthesis and post-translational modifications. Therefore, an existing method is used to estimate TF activity based on the gene expression profile in a tissue sample (see Appendix A for details). Such a method is not suitable for estimating the activity of PD-L1. Hence, gene expression is used directly as a proxy for the activity of PD-L1 in a sample.

In biological modeling, it is common to assume an “OR” relation between TFs and target gene expression, i.e., “if TF A or TF B or TF C is active, then gene X is likely expressed”. When investigating which TF act together inside a complex, the natural analogue is to assume an “AND” gate. That is, “if TF A and TF B are active, then gene X is likely expression”. In reality, different complexes may lead to a gene’s transcription (expression), so a more fitting model is an “OR-of-ANDs”. Denote an “OR” by $\vee$ and an “AND” by $\wedge$. An example is

$$\begin{aligned} &(\text{TF A is active} \wedge \text{TF B is active}) \vee (\text{TF C is active} \wedge \text{TF D is active}) \\ &\implies \text{Gene X is expressed} \end{aligned}$$

From a mathematical perspective, this leads us to the subject of “ruleset learning”. Ruleset learning is concerned precisely with learning a formula of this form from data. A major advantage of rulesets over, for instance, neural networks, is that rulesets are easily interpretable.

There exist related data mining techniques known for their interpretability, like decision trees. Rulesets and decision trees are mathematically related; one can always be expressed in the form of the other. However, they differ conceptually and in terms of expressiveness. A relatively simple ruleset can be equivalent to a large, redundant decision tree and vice versa. Notably, a decision trees holds certain hierarchy; conditions checked higher up the tree are “more important”. The root node, for instance, plays a pivotal role in all decisions. We are looking for a diverse set of relevant conditions. It is not obvious that activity of one TF individually should govern the entire decision process. Therefore, while substantial literature exists on optimal decision trees ([18]), we do not consider decision trees suitable for this problem. Another related technique is optimal rule list mining, which was studied in [2]. A rule list (or decision list) is an ordered list of if-statements, evaluated sequentially to make

a prediction. For instance,

if (TF A is active $\wedge$ TF B is active) **then** predict “Gene X expressed”,
 else if (TF C is active) **then** predict “Gene X expressed”

Again there is a hierarchy; later rules assume that earlier rules were false.

Therefore, we build upon earlier research into optimal rule sets, studied in [11] and later in [17]. We will study whether rule sets can be effective in predicting gene expression from TF activity scores in the context of bulk transcriptomics. Furthermore, we will investigate whether the rule sets describe an intrinsic phenomenon in the data. Precisely, we study whether the rule sets found are consistent and generalize to unseen data.

1.1 Data

The gene expression datasets used are Broad and Sanger. These were extracted from the RNA-Seq 2022-06-24 data collection, downloaded from the Cell Model Passports hub[21, 9]. This collection contains RNA transcript counts of thousands of genes, including PD-L1, for hundreds of cancer cell line samples. We will refer to an RNA transcript count of a gene as simply “gene expression”. Each datapoint, corresponding to one cancer cell line, is referred to as simply a “sample”. Combined, the datasets contain 1291 samples. This is few and eliminates the possibility of using methods like neural networks.

For each sample, the tissue status and tissue type is recorded in the dataset. The samples consist of a wide variety of cancer types found in various types of tissue. Tissue status among the samples is not homogenous. Sample counts per dataset and tissue status are given in Table 1.1.

Table 1.1: Sample counts by dataset and tissue type.

Dataset	Tumor	Metastasis	Papiloma	Normal	Unknown	Total
Broad	507	272	1	1	68	849
Sanger	240	113	0	6	83	442
Total	747	385	1	7	151	1291

Out of all candidate TF, a total of 18 were selected to be of interest. This selection was based on literature, in particular whether a TF can physically bind to the relevant regulatory sequence involved in PD-L1 expression. Descriptive statistics for the corresponding TF activities and PD-L1 expression in both Broad and Sanger can be found in Table A.1 and Table A.2, respectively.

The cross-correlation matrix of TF activities is depicted in Figure A.1a. The cross-correlation matrix of the gene expressions (used to calculate TF activities)

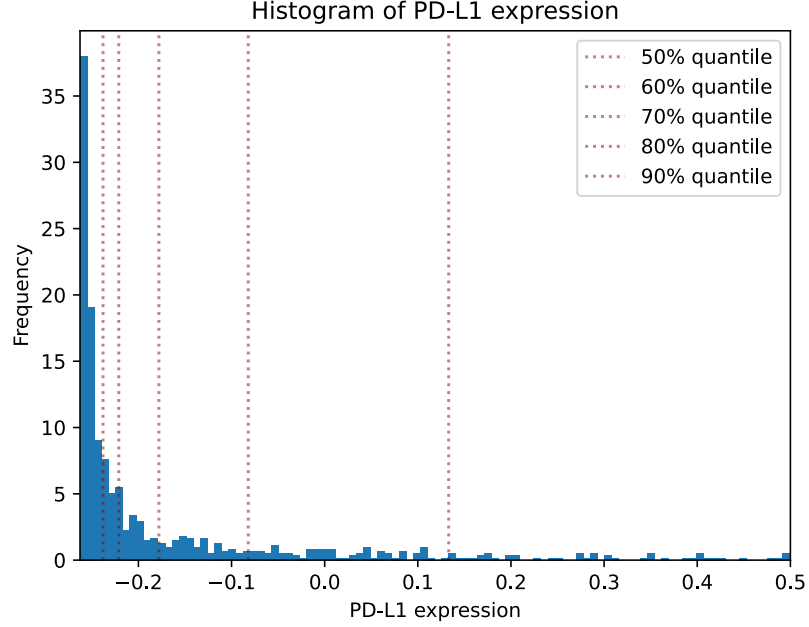


Figure 1.1: Histogram of PD-L1 expression in Broad dataset. Increasing quantiles are also depicted from left to right using vertical lines.

is depicted in Figure A.1b. Notably, TF activities are highly correlated, while gene expressions are not. Both figures also include correlation with PD-L1 expression on the bottom row.

PD-L1 expression for Broad and Sanger dataset combined was normalized to mean 0 and standard deviation 1, The histogram of PD-L1 expression of Broad samples is depicted in Figure 1.1. It shows that the majority of observations clusters around the minimum. Note that the plot starts at the minimum observed expression value and is capped at an expression value of 0.5, while the maximum observed PD-L1 expression is 3.55.

Broad serves as the training dataset. Cross-validation will be used on splits of Broad to evaluate model performance. Sanger serves as the holdout dataset and will be used only once, at the very end (Chapter 6).

Chapter 2

Boolean model

We consider an approach from literature that is capable of learning logical rules from binary data. It is a binary classification approach, meaning the input is a set of points, their binary features and for each point a target boolean. The output is a boolean formula in Disjunctive Normal Form (DNF, OR-of-ANDs).

A DNF is a disjunction (OR) of one or more conjunctions (AND). We refer to a conjunction as simply a “rule”. A rule consists of one more clauses. In the boolean setting, a clause is a literal, meaning a feature or its negation. For instance, if A is a feature representing whether TF A is active, then the corresponding clauses are A and $\neg A$. If A and B are features, then an example of a rule is

$$(A \wedge \neg B),$$

meaning “ A and not B ”. A disjunction of rules is what we will call a ruleset. For instance,

$$(A \wedge B) \vee (B \wedge \neg C),$$

meaning “Either A and B , or B and not C ”, is a ruleset. In practice, negated literals do not need to be modeled separately, as a feature’s negated values can be added as a separate feature during pre-processing.

The goal of the approach is to find a ruleset describing the points that have target value “True”. We say those points are positive points, whereas points with a target value of “False” are called negative points. Negative points should not satisfy the ruleset. Hence, the ruleset found should evaluate to “True” for positive points and “False” for negative points. In other words, the ruleset is a binary classifier and the target values are the (binary) class labels. The objective of this approach is to find a ruleset that maximizes accuracy, that

is to maximize

$$\frac{\text{\# correctly classified points}}{\text{\# points}},$$

where “#” stands for “number of”. Since the number of points is constant, we may multiply by it and maximize

$$\text{\# correctly classified points}.$$

This is equivalent to minimizing

$$\text{\# points} - \text{\# correctly classified points},$$

as we simply negate and add a constant. This is equal to

$$\text{\# misclassified points}.$$

Separating positive and negative points, we find this equal to

$$\text{\# false negatives} + \text{\# false positives}. \quad (2.1)$$

Hence, maximizing accuracy and minimizing this quantity are equivalent. In [17], it turns out to be easier to minimize a similar quantity, namely

$$\text{\# false negatives} + \text{\# times a rule satisfies a negative point}. \quad (2.2)$$

For every rule in the ruleset, it counts how many negative points it covers. Hence, we don’t just count whether a negative point is misclassified, but we count how often. It follows that (2.2) is at least as large as (2.1). If the rules do not “overlap” much, then (2.2) is close to (2.1). This method has been shown to work well on a variety of binary datasets.

We wish to apply this method to TF data and PD-L1 expression. Conceptually, the target boolean should represent whether the target gene was “Expressed” (positive) or “Not expressed” (negative). A simple and intuitive way to do this is via thresholding. That is, the target considered is

$$\text{PD-L1 expression} \geq \text{PD-L1 threshold}.$$

A natural choice for threshold is a quantile level observed in the data, i.e. the 0.5 quantile, which is the median. However, which quantile should be chosen is unclear, as there is no unanimous definition for a gene being expressed or not expressed. Figure 1.1 suggests any of the depicted quantiles could be a valid threshold.

A feature should represent whether a TF was “Active” or “Inactive” Again, these states do not have a clear definition and the data is continuous. For features, we have the freedom to add several binary features per TF, for instance

$$\begin{aligned} \text{TF} &\geq 0.1 \text{ quantile}, \\ \text{TF} &\geq 0.2 \text{ quantile}, \\ &\vdots \\ \text{TF} &\geq 0.9 \text{ quantile}. \end{aligned}$$

These binary features are meant to represent different activity levels, e.g., “at least somewhat active”, “at least moderately active”, et cetera.

In this chapter, we first cover the mathematical background of the method (Section 2.1 and Section 2.2). Then we briefly discuss our implementation and compare its performance to the performance of an implementation from literature (Section 2.3). Next, we discuss the methodology for a first experiment on TF data (Section 2.4). Finally, we discuss the results (Section 2.5). A reader who is not interested in the mathematics and the implementation may skip to Section 2.4.

2.1 Model description

We consider the supervised binary classification setting studied in [17]. What follows is a recap that closely follows this paper. Given is a training set of n points, denoted by $\mathcal{I}$, from an underlying (unknown) distribution. Each point has m features from set $\mathcal{J}$, plus a label

$$(\mathbf{X}_i, y_i), \quad i \in \mathcal{I} = \{1, \dots, n\}.$$

In this setting, feature values $\mathbf{X}_i \in \{0, 1\}^{\mathcal{J}}$ and labels $y_i \in \{-1, 1\}$ for all $i \in \mathcal{I}$. A point i is said to possess feature j if $X_{ij} = 1$, where X_{ij} is the j -th element of vector $\mathbf{X}_i$. A point i is said to be positive if $y_i = 1$ and negative otherwise.

Theoretically, binary features are not restrictive. For example, categorical features can be expressed as binary features via one-hot encoding. Real-valued features can be discretized into a number of binary features via thresholds, i.e., binary features reflecting $X_{ij} \leq v$ or $X_{ij} \geq v$ may be introduced for various v . The number of features can grow rapidly with this approach.

The objective of this model is to learn a ruleset as defined earlier. Mathematically, define the set of possible clauses as $\mathcal{L} = \mathcal{J}$. A rule is a subset of $\mathcal{L}$. The candidate rule set is given by $\mathcal{K} = 2^{\mathcal{L}}$, i.e. the power set of $\mathcal{L}$. Let $\mathcal{K}_i$ be the set of rules satisfied by point i , so

$$\mathcal{K}_i = \left\{ k \in \mathcal{K} : \bigwedge_{j \in k} X_{ij} \right\}, \quad i \in \mathcal{I}. \quad (2.3)$$

In other words, $\mathcal{K}_i$ is the set of rules that cover point i . Observe that $\emptyset \in \mathcal{K}$ and that the conjunction in (2.3) is always true for $k = \emptyset$. So $\emptyset \in \mathcal{K}_i$ for any $i \in \mathcal{I}$. Therefore, the “empty rule” $\emptyset$ covers all points under this definition.

Definition 2.1.1 (DNF Classifier, [17]). Given a rule set $S \subseteq \mathcal{K}$, the DNF classifier is defined as follows:

$$\hat{y}_S(\mathbf{X}_i) = \begin{cases} 1, & \text{if } |\mathcal{K}_i \cap S| > 0, \\ 0, & \text{else.} \end{cases}$$

There exist 2^m possible rules, i.e. $|\mathcal{K}| = 2^m$. Therefore, as m grows, enumerating all possible rules can quickly become intractable. In practice, one may wish to restrict S , for example by limiting the number of clauses in a rule to D . This restricts the number of possible rules to

$$\sum_{d=0}^D \binom{m}{d}.$$

The aim is to minimize the 0-1 classification error, which counts the number of datapoints that are misclassified by a rule set S . Hence, it is the sum of the 0-1 loss of each datapoint.

Definition 2.1.2 (0-1 loss and 0-1 classification error). The *0-1 loss* of point i subject to ruleset S is

$$\ell_i^{01}(S) = \begin{cases} \mathbb{I}(|\mathcal{K}_i \cap S| = 0), & \text{if } y_i = 1, \\ \mathbb{I}(|\mathcal{K}_i \cap S| > 0), & \text{if } y_i = -1, \end{cases}$$

where $\mathbb{I}(\mathcal{A})$ is the indicator function (i.e., $\mathbb{I}(\mathcal{A}) = 1$ if $\mathcal{A}$ is true and 0 otherwise). The *0-1 classification error* of ruleset S is

$$\ell^{01}(S) = \sum_{i \in \mathcal{I}} \ell_i^{01}(S).$$

Closely related is the accuracy, which is the fraction of correctly classified points.

Definition 2.1.3 (Accuracy). The *accuracy* of a ruleset S is

$$\text{Accuracy}(S) = \frac{1}{n} |\{i \in \mathcal{I} : \hat{y}_S(\mathbf{X}_i) = y_i\}|.$$

Minimizing the 0-1 classification error is equivalent to maximizing the classification accuracy, as illustrated in the introduction to this chapter.

We now formulate the problem of finding a ruleset with minimum 0-1 classification error as an Integer Program (IP). Partition the points into two sets, the positive class and the negative class:

$$\mathcal{P} = \{i \in \mathcal{I} : y_i = 1\} \text{ and } \mathcal{N} = \{i \in \mathcal{I} : y_i \neq 1\}.$$

Let $w_k \in \{0, 1\}$ be a variable indicating if rule $k \in \mathcal{K}$ is used in the rule set. Let $\zeta_i \in \{0, 1\}$ be a variable indicating if point $i \in \mathcal{P} \cup \mathcal{N}$ is misclassified. The IP is given by

$$\min \quad \sum_{i \in \mathcal{P}} \zeta_i + \sum_{i \in \mathcal{N}} \zeta_i \quad (2.4a)$$

$$\text{s.t.} \quad \zeta_i + \sum_{k \in \mathcal{K}_i} w_k \geq 1, \quad \forall i \in \mathcal{P}, \quad (2.4b)$$

$$w_k \leq \zeta_i, \quad \forall i \in \mathcal{N}, k \in \mathcal{K}_i, \quad (2.4c)$$

$$w \in \{0, 1\}^{\mathcal{K}}, \quad \zeta \in \{0, 1\}^{\mathcal{P} \cup \mathcal{N}}. \quad (2.4d)$$

The objective of this IP is exactly the 0-1 classification error. To see that this IP is correct, consider first (2.4b). Let $i \in \mathcal{P}$. If $w_k = 1$ for some $k \in \mathcal{K}_i$, then i is covered by the ruleset. Indeed, then $\zeta_i = 0$ is valid and optimal. If not, then i is not covered and indeed $\zeta_i = 1$ to satisfy the constraint. So ζ_i correctly represents whether i is misclassified. Now consider (2.4c). If $w_k = 1$ for some $k \in \mathcal{K}_i$, then i is covered and (2.4c) implies $\zeta_k = 1$. Otherwise, $\zeta_i = 0$ is valid and optimal. So ζ_i correctly represents whether i is misclassified.

This formulation has one integer variable per point and multiple constraints per negative datapoint. For a large number of points, the IP becomes prohibitively large. In many practical classification tasks, the number of positive points is small compared to the total. To limit the size of the IP, the Hamming loss may be considered. In the Hamming loss, we don't count the number of negative points that are misclassified, but we count the total number of times a negative point is misclassified by a rule.

Definition 2.1.4 (Hamming loss [17]). The *Hamming loss* of point i under rule-set S is given by

$$\ell_i^h(S) = \begin{cases} \mathbb{I}(|\mathcal{K}_i \cap S| = 0), & \text{if } y_i = 1, \\ |\mathcal{K}_i \cap S|, & \text{otherwise.} \end{cases}$$

It has been illustrated that Hamming loss is a good alternative to 0-1 loss in practical settings [17].

Furthermore, it is useful in practice to restrict the complexity of a rule set. Define the complexity of a rule k as

$$c_k = 1 + |k|, \quad (2.5)$$

so one plus the number of features in the rule. The complexity of a rule set $K \subseteq \mathcal{K}$ is given by $\sum_{k \in K} c_k$, which we bound by C .

The new IP that uses Hamming loss becomes

$$\min \quad \sum_{i \in \mathcal{P}} \zeta_i + \sum_{i \in \mathcal{N}} \sum_{k \in \mathcal{K}_i} w_k \quad (2.6a)$$

$$\text{s.t.} \quad \zeta_i + \sum_{k \in \mathcal{K}_i} w_k \geq 1, \quad \forall i \in \mathcal{P}, \quad (2.6b)$$

$$\sum_{k \in \mathcal{K}} c_k w_k \leq C, \quad (2.6c)$$

$$w \in \{0, 1\}^{\mathcal{K}}, \zeta \in \{0, 1\}^{\mathcal{P}}. \quad (2.6d)$$

For a positive point i , the justification given for (2.4) still holds. For a negative point i , there is no ζ_i , but instead the number of points wrongly covering it are counted directly in the objective.

The number of possible rules $|\mathcal{K}|$ grows quickly as m grows. Therefore, column generation is used to solve the LP relaxation of this problem. A simple approach for obtaining an integer solution is employing a price-then-branch heuristic. In this technique, the relaxed master problem (RMP) is first solved to optimality. Next, the restricted integer master problem, with only the previously generated columns, is solved to optimality using a standard IP solver. While it is possible to solve the problem exactly via a branch-and-price approach, this is harder to implement, it is slower, and it is unclear whether it improves the quality of the output significantly. In some cases, the price-then-branch heuristic can reveal that an exact solution is only marginally better in terms of objective.

2.2 Pricing problem

Let $(\mu, \lambda) \in \mathbb{R}_+^{|\mathcal{P}|} \times \mathbb{R}_+$ be an optimal dual solution to RMP, where dual values μ, λ correspond to constraints (2.6b) and (2.6c) respectively. The reduced cost of a variable w_k , associated with a rule k not yet in the master problem, can be expressed as

$$\hat{\rho}_k = \sum_{i \in \mathcal{N}} \mathbb{I}(k \in \mathcal{K}_i) - \sum_{i \in \mathcal{P}} \mu_i \mathbb{I}(k \in \mathcal{K}_i) + \lambda c_k. \quad (2.7)$$

For the derivation we refer to Section B.1. If $\hat{\rho}_k < 0$, then adding rule k to the relaxed master problem can improve its objective. If $\hat{\rho}_k \geq 0$ for all $k \in \mathcal{K}$, we have found an optimal LP solution. Deciding this is the pricing problem.

To solve the pricing problem, we introduce the variables $z_l \in \{0, 1\}, l \in \mathcal{L}$ representing whether the rule includes clause l . We also introduce the variables $\delta_i \in \{0, 1\}, i \in \mathcal{I}$ denoting whether point i is covered by the rule. The complexity of a rule can be expressed as

$$c(z) = 1 + \sum_{l \in \mathcal{L}} z_l, \quad z \in \{0, 1\}^{\mathcal{L}}. \quad (2.8)$$

Define

$$S_i = \{l \in \mathcal{L} : X_{ij} = 0\}, \quad i \in \mathcal{I}. \quad (2.9)$$

Hence, the set S_i contains all clauses (binary features in this setting) that point i does not satisfy (possess). The pricing problem asks whether there exists a $k \in \mathcal{K}$ such that $\rho_k < 0$. It may be solved by minimizing (2.7). The corresponding IP, as described in [17], is given by

$$\min \quad \sum_{i \in \mathcal{N}} \delta_i - \sum_{i \in \mathcal{P}} \mu_i \delta_i + \lambda c(z) \quad (2.10a)$$

$$\text{s.t.} \quad D\delta_i + \sum_{j \in S_i} z_j \leq D, \quad i \in \mathcal{I}, \quad (2.10b)$$

$$\delta_i + \sum_{j \in S_i} z_j \geq 1, \quad i \in \mathcal{I}, \quad (2.10c)$$

$$\sum_{j \in \mathcal{J}} z_j \leq D, \quad (2.10d)$$

$$z \in \{0, 1\}^{\mathcal{J}}, \delta \in \{0, 1\}^{\mathcal{I}}. \quad (2.10e)$$

If $\delta_i = 1$, then by (2.10b), no clause cutting off i can be chosen, so i must be covered by the rule. If $\delta_i = 0$, then by (2.10c), some clause cutting off i must be chosen, so i cannot be not covered by the rule. Hence, δ_i reflects whether i is covered. Therefore, the objective (2.10a) matches the reduced cost (2.7). So the IP finds a rule with minimum reduced cost.

Constraints (2.10b) can also be expressed as

$$\delta_i + z_j \leq 1, \quad i \in \mathcal{I}, j \in S_i. \quad (2.11)$$

This formulation may be favorable in terms of solver efficiency, as it provides a sharper LP relaxation. These two formulations pose a tradeoff; while (2.11) implies the (2.10b), it is also a larger family of constraints which could significantly slow down the LP solver. Furthermore, it is unclear how the pre-solvers employed modern IP solvers handle these formulations within this IP. This makes it hard to predict which formulation is favorable.

2.3 Implementation

A custom solver is implemented in C++ using the SCIP 10 framework with SoPlex 8 [12]. Note that the implementation of [17] is available on GitHub [16]. That implementation is written in Python and uses proprietary software Gurobi. Our implementation is fully open source and also available on GitHub . This custom solver includes an efficient heuristic pricer and was designed to support the extensions discussed in the next chapters. The custom solver has two modes:

Upload to GitHub and cite

1. **Full:** Constructs and solves the full master problem (2.6), with all possible rules (columns),
2. **Column generation (CG):** Performs the price-then-branch approach.

The Full mode can only realistically be solved for datasets with small $\mathcal{I}$ or small D . It can serve to verify, to some extent, the performance and validity of the column generation approach. The CG mode can use up to three pricers:

1. **Iterative:** Iterates over all possible rules and computes reduced cost naively,
2. **Exact:** Solves MIP (2.10) to optimality,
3. **Heuristic:** Employs a beam search.

The Exact pricer produces a single rule minimizing reduced cost (unless exact pricing is interrupted early). The other pricers can be configured to return up to some maximum of rules, and to return a “diverse” collection of rules.

We now discuss the Heuristic pricer and the efforts taken to implement it efficiently. The Heuristic pricer builds a tree, in which every node is a rule and each layer represents rules of a given size.

The root node is the empty rule $\emptyset$, covering all points. The Heuristic pricer starts by constructing all children of the root node and computing the reduced cost. Then the nodes in this layer are pruned: only the nodes with the lowest reduced cost are kept. The beam width at this depth of the tree determines the number of nodes kept. This process repeats: in iteration ν , the children of each node at depth $\nu - 1$ are constructed. Next, the number of nodes at level ν are reduced to the beam width of layer ν .

Computing a node’s reduced cost naively costs $\Theta(n)$ operations. For large datasets, this can be slow. We note that there exists overlap between the computations of parents and those of their descendents. We may exploit this overlap. Let $k_{\text{parent}}, k_{\text{child}}$ be the rules corresponding to some parent and child node, respectively. Recall that the reduced cost

$$\hat{\rho}_{k_{\text{child}}} = \sum_{i \in \mathcal{N}} \mathbb{I}(k_{\text{child}} \in \mathcal{K}_i) - \sum_{i \in \mathcal{P}} \mu_i \mathbb{I}(k_{\text{child}} \in \mathcal{K}_i) + \lambda c_{k_{\text{child}}}. \quad (2.12)$$

Define $\mathcal{I}_k$ as the set of points covered by rule k , so

$$\mathcal{I}_k := \{i \in \mathcal{I} : k \in \mathcal{K}_i\} = \left\{ i \in \mathcal{I} : \bigwedge_{j \in k} X_{ij} \right\}, \quad k \in \mathcal{K}. \quad (2.13)$$

Define $\mathcal{P}_k$ and $\mathcal{N}_k$ analogously. Observe that

$$k_{\text{parent}} \notin \mathcal{K}_i \implies k_{\text{child}} \notin \mathcal{K}_i, \quad i \in \mathcal{I}.$$

In other words, a point that does not satisfy the parent rule can never satisfy the child rule, because the child rule is more restrictive. Then

$$\mathcal{I}_{k_{\text{child}}} \subseteq \mathcal{I}_{k_{\text{parent}}}.$$

The analogous property holds for $\mathcal{P}_k$ and $\mathcal{N}_k$. It follows that

$$\begin{aligned} \hat{\rho}_{k_{\text{child}}} &= |\mathcal{N}_{k_{\text{child}}}| - \sum_{i \in \mathcal{P}_{k_{\text{child}}}} \mu_i + \lambda c_{k_{\text{child}}} \\ &= |\mathcal{N}_{k_{\text{parent}}}| - |\mathcal{N}_{k_{\text{parent}}} \cap \mathcal{N}_{k_{\text{child}}}| \\ &\quad - \left(\sum_{i \in \mathcal{P}_{k_{\text{parent}}}} \mu_i - \sum_{i \in \mathcal{P}_{k_{\text{parent}}} \cap \mathcal{P}_{k_{\text{child}}}} \mu_i \right) \\ &\quad + \lambda(c_{k_{\text{parent}}} + 1) \\ &= \hat{\rho}_{k_{\text{parent}}} - |\mathcal{N}_{k_{\text{parent}}} \cap \mathcal{N}_{k_{\text{child}}}| + \sum_{i \in \mathcal{P}_{k_{\text{parent}}} \cap \mathcal{P}_{k_{\text{child}}}} \mu_i + \lambda. \end{aligned} \tag{2.14}$$

To use this equation, we may store $\mathcal{P}_k$, $\mathcal{N}_k$ and the reduced cost for each node. In our implementation, the sets are stored as bitmaps, reducing memory footprint and allowing for efficient iteration over

$$\mathcal{P}_{k_{\text{parent}}} \cap \mathcal{P}_{k_{\text{child}}} \quad \text{and} \quad \mathcal{N}_{k_{\text{parent}}} \cap \mathcal{N}_{k_{\text{child}}}.$$

The cost of this approach is memory. For every node kept in the search tree, these bitmaps need to be stored. To avoid unnecessary memory use, these bitmaps are computed after pruning, meaning these bitmaps are never created for pruned nodes.

The heuristic pricer can also be configured to do a “deep” search. In this case, pruning is not done per layer but per parent node. Setting the “beam width” (which is no longer a beam width, but a parent’s degree) to m or higher, this effectively means the heuristic pricer does enumeration of all rules up to size D , only through this memory-heavy but computationally-efficient approach. Depending on the available memory, the number of points number of features and D , the iterative pricer may be preferred.

We compiled our implementation on Eindhoven University of Technology’s High Performance Cluster (HPC), allowing us to perform many experiments simultaneously. The HPC is a shared computer. Hence, we cannot guarantee experiments are always run under the same load conditions. Furthermore, we did not specifically request the same exact hardware to be used for each experiment. Therefore, running times are not to be compared between experiments.

To test whether our implementation performs similarly to the implementation used in [17] we perform a benchmark. We test our implementation on datasets marked “small” in the paper. Some of the datasets could not be traced

back. These datasets were downloaded from the UCI Machine Learning Repository [14]. For the others, we set C to the mean value reported in the paper, which was reportedly the average C yielding the best test accuracy. For continuous variables, binarization is performed with 9 quantiles. Categorical variables are one-hot encoded. We employ the heuristic pricer with beam widths (100, 50, 50, 20), identical to the original paper. The exact pricer is used when the heuristic pricer does not find any rule with negative reduced cost. The pricing stage is limited to 2 hours. The branching stage is limited to 1 hour, but in practice this limit is never reached. Like in the original paper, 10-fold stratified cross validation is used.

Table 2.1: Results for the benchmark between our implementation and the implementation of [16]. Metrics are reported in “[Mean] ([Standard deviation])” format. “Accuracy” is abbreviated to “Acc.”.

Dataset	C	Solve time (s)	Test acc. (%)	Test acc. (%) in [17]	Train acc. (%)
ILPD [22]	11	6 529 (1 191)	69.4 (3.0)	69.6 (1.2)	78.2 (0.3)
WDBC [26]	14	6 859 (519)	93.3 (3.8)	94.0 (1.2)	99.0 (0.2)
banknote [19]	25	105 (35)	99.3 (0.7)	99.1 (0.3)	99.8 (0.1)
heart [13]	12	649 (303)	72.7 (7.7)	78.9 (2.4)	87.6 (0.9)
ionosphere [23]	13	7 056 (456)	88.3 (5.1)	90.0 (1.8)	95.8 (0.6)
tic-tac-toe [1]	32	1 689 (567)	100.0 (0.0)	100.0 (0.0)	100.0 (0.0)
transfusion [27]	6	14 (14)	79.3 (3.9)	77.9 (1.4)	79.4 (0.5)

The results are given in Table 2.1. Test accuracies are similar for both implementations. Not evident from the table is that our implementation is slower. In some cases, the time limit of the pricing stage is reached. We attribute this to the speed at which the exact pricer solves the pricing problem.

2.4 Methodology

As a first experiment, we use the boolean model on the full Broad dataset, binarized in pre-processing via quantile thresholding. The idea is to allow a moderate complexity, meaning a reasonably high C , and study the performance of the ruleset in terms of accuracy. At this point we are not interested in preventing overfitting, but merely the existence of a performant ruleset.

Let v be a vector containing measurements. Define $Q_v: [0, 1] \rightarrow \mathbb{R}$ as the quantile function (or percentile function) over these measurements. The quantile function uses linear interpolation when given an unobserved quantile level as input value. The target for point i is

$$y_i \geq Q_y(Q_y),$$

where $Q_y \in [0, 1]$ is the chosen target quantile level.

Note that the choice of Q_y determines the balance of positive and negative samples in the dataset. For $Q_y = 0.5$, the target is simply whether a point's PD-L1 expression is at least the median. By construction, this leads to a (nearly) perfectly balanced dataset. For $Q_y = 0.9$, for instance, the dataset is heavily imbalanced, as only a tenth of the samples are positive. It is important to keep this in mind when evaluating models. Notably, the empty ruleset $\emptyset$ achieves an accuracy of Q_y for all $Q_y \in [0, 1]$. This is because the empty ruleset covers no points and a fraction of Q_y points is negative by construction. Hence, a ruleset for $Q_y = 0.9$, for instance, should show at least 0.9 accuracy, and preferably much higher. Other performance measures that may be considered in the case of imbalanced datasets are precision and recall. Precision measures the fraction of positively classified points is truly positive. Recall measures the fraction of positive points that was positively classified.

Add a mathematical definition? I don't think it would add anything

Denote by $n_{\text{quantiles}}$ the number of thresholds to use for each TF. Then for some TF with observed activity vector $\mathbf{a}$, we consider the binary features

$$\begin{aligned} a_i &\geq Q_{\mathbf{a}} \left(\frac{1}{n_{\text{quantiles}} + 1} \right), \\ a_i &\geq Q_{\mathbf{a}} \left(\frac{1}{n_{\text{quantiles}} + 1} \right), \\ &\vdots \\ a_i &\geq Q_{\mathbf{a}} \left(\frac{n_{\text{quantiles}}}{n_{\text{quantiles}} + 1} \right). \end{aligned} \tag{2.15}$$

Here, $\cdot$ is a placeholder for a point i . All these binary features combined form $\mathbf{X}$.

We consider two variants of the experiment. In the first variant, only the binary features described in (2.15) are used. This means that rules with more than one TF represent simultaneous activity of these TF. The second variant also adds the reverse binary features, corresponding to the upper-bounds

$$\begin{aligned} a_i &\leq Q_{\mathbf{a}} \left(\frac{1}{n_{\text{quantiles}} + 1} \right), \\ a_i &\leq Q_{\mathbf{a}} \left(\frac{1}{n_{\text{quantiles}} + 1} \right), \\ &\vdots \\ a_i &\leq Q_{\mathbf{a}} \left(\frac{n_{\text{quantiles}}}{n_{\text{quantiles}} + 1} \right). \end{aligned} \tag{2.16}$$

A rule can then be a combination of lower and upperbounds. Assume for now that a TF appears at most once in a rule. Then a rule can represent simultaneous activity of some TF and inactivity of others. An upper-bound can be interpreted as an “interfering” TF that, when present, prevents other TF from acting together.

We solve the model via column generation for

$$C = 20, \quad D = 2, \quad n_{\text{quantiles}} = 1, 3, 5, 7, 9.$$

Recall that, in accordance with [17], a rule’s complexity is 1 plus the number of features in it. As such, a ruleset complexity of 20 is equivalent to five rules of size three. A ruleset complexity of 18 is equivalent to six rules of size two.

We employ a heuristic pricer with beam widths (1000, 500, 500). We also employ the exact pricer. We limit pricing (the finding of rules) to 23 hours and branching to 1 hour.

2.5 Results

Performance for the first variant is shown in Figure 2.1. Performance barely increases as $n_{\text{quantiles}}$ increases, though there appears to be some effect for lower Q_y . Importantly, performance for low Q_y is moderate, while performance for high Q_y is bad. For $Q_y = 0.9$, for instance, accuracy is close to 0.9, while an accuracy of 0.9 can be achieved with an empty ruleset. The same is observed for $Q_y = 0.8$. For $Q_y = 0.5$, however, an accuracy of over 0.70 is achieved.

Performance for the second variant is shown in Figure 2.2. The analysis is analogous. The extra features yield modest increase in performance overall, suggesting that lower-bounds may be important for finding relevant rulesets.

This method also gives results on the best possible rulesets that exist. Its conclusion that there do not exist meaningfully better rulesets, in the sense that no significantly higher accuracy can be reached. For further details we refer the reader to Section D.1.

Based on these results, choosing $Q_y = 0.5$ or $Q_y = 0.6$ is preferred. However, the results are underwhelming, compared to the accuracies that can be achieved on benchmark datasets. While it may be possible to find consistent rulesets describing relevant phenomena, rulesets of this form with moderate complexity ($C = 20$) are not good predictors for thresholded PD-L1 expression.

Nevertheless, we are interested in the behavior of the model as we restrict complexity. How much does performance drop? What do simple rulesets look like? Are they consistent among splits of the dataset?

To study these results, we will tailor the model to thresholds to speed up computation and to set the scene for future extensions.

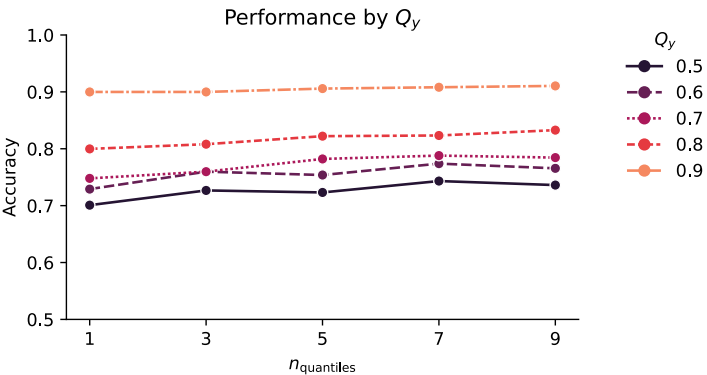


Figure 2.1: Ruleset performance for the first variant of the experiment, only considering binary features (2.15).

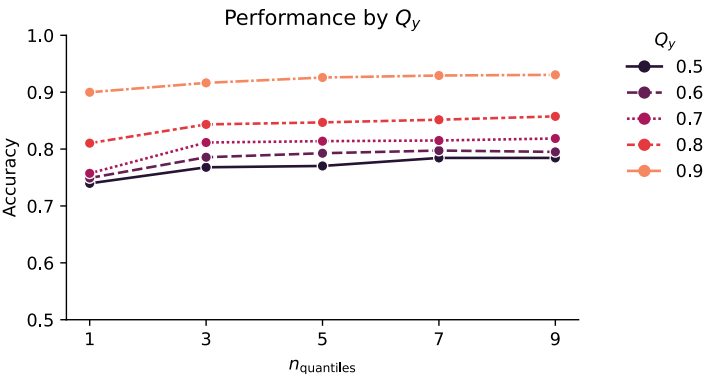


Figure 2.2: Ruleset performance for the first variant of the experiment, considering both features (2.15) and (2.16).

Chapter 3

Threshold model

In Chapter 2, a first experiment on TF activities and PD-L1 expression was conducted using a method designed for binary data. One drawback of this method is that, after binarization, information on which binary features belong to which TF is lost. This information could be used in the model, for instance to speed up computation.

The TF activity data is continuous. Instead of pre-binarizing via thresholding and employing the boolean-featured model, we adjust the boolean model to consider threshold clauses on continuous features. This can not only speed up solving, it sets the stage for future extensions that attempt to exploit the continuous nature of the TF data. This model, which we call the threshold model, is our contribution.

The structure of this chapter follows closely that of Chapter 2. A reader who is not interested in the mathematics and implementation of the method but rather the next experiments and their results can skip to Section 3.3.

3.1 Model description

Recall the boolean model as described in Section 2.1. In this setting, feature values are continuous, so $X_j \in \mathbb{R}^{\mathcal{J}}$, $j \in \mathcal{J}$. A clause represents a lower-bound, and consists of a feature and a threshold. An upper-bound is equivalent to a lower-bound on the negated feature, i.e.

$$X_{,j} \leq \alpha \iff -X_{,j} \geq -\alpha.$$

Hence, without loss of generality, only lower-bounds are considered. We denote the set of possible clauses by $\mathcal{L} \subseteq \mathcal{J} \times \mathbb{R}$. The candidate rule set remains

$\mathcal{K} = 2^{\mathcal{L}}$, the power set of $\mathcal{L}$. In this setting,

$$\mathcal{K}_i = \left\{ k \in \mathcal{K} : \bigwedge_{(j,\alpha) \in k} X_{ij} \geq \alpha \right\}, \quad i \in \mathcal{I}. \quad (3.1)$$

The master problem formulation (2.6) remains unchanged.

While a threshold can theoretically be any value, it suffices to consider only thresholds $\{X_{ij}\}_{i \in \mathcal{I}}$ for any feature j . Any other threshold yields a clause that is equivalent, at least in training, to one already considered. For example, if

$$\mathcal{J} = \{a\}, \mathcal{I} = \{1, 2\} \text{ and } X_{1,a} = 1, X_{2,a} = 2,$$

the only relevant lower-bounds are

$$X_{\cdot,a} \geq 1 \text{ and } X_{\cdot,a} \geq 2.$$

Hence, we consider

$$\mathcal{L} = \{(j, X_{ij}) : j \in \mathcal{J}, i \in \mathcal{I}\}. \quad (3.2)$$

The pricing problem IP is similar to (2.10). The difference is that we now speak of $z_l, l \in \mathcal{L}$, and we define S_i as

$$S_i = \{(j, \alpha) \in \mathcal{L} : X_{ij} < \alpha\}, \quad i \in \mathcal{I}. \quad (3.3)$$

Hence, S_i the set of clauses that cut off point i . The IP is given by

$$\min \quad \sum_{i \in \mathcal{N}} \delta_i - \sum_{i \in \mathcal{P}} \mu_i \delta_i + \lambda c(z) \quad (3.4a)$$

$$\text{s.t.} \quad D\delta_i + \sum_{l \in S_i} z_l \leq D, \quad i \in \mathcal{I}, \quad (3.4b)$$

$$\delta_i + \sum_{l \in S_i} z_l \geq 1, \quad i \in \mathcal{I}, \quad (3.4c)$$

$$\sum_{l \in \mathcal{L}} z_l \leq D, \quad (3.4d)$$

$$\delta \in \{0, 1\}^{\mathcal{I}}, z \in \{0, 1\}^{\mathcal{L}}. \quad (3.4e)$$

We impose another constraint in this setting: the use of at most one clause per feature $j \in \mathcal{J}$. This constraint can be written as

$$\sum_{(j', \alpha) \in \mathcal{L} : j' = j} z_{(j', \alpha)} \leq 1, \quad j \in \mathcal{J}. \quad (3.5)$$

If negated feature values are added as separate features to model upper-bounds, then (3.5) does not prevent a lower-bound and an upper-bound on the same feature from being used in one rule.

raise once more

Note that $|\mathcal{L}| = |\mathcal{I}||\mathcal{J}|$. In our case, $|\mathcal{L}| \approx 800 \cdot 20 \approx 16000$. This is rather large, causing $|\mathcal{K}|$ to explode. An alternative is to choose

$$\mathcal{L} = \left\{ \left(j, Q_{\mathbf{X}_j}(q) \right) : j \in \mathcal{J}, q \in \mathbf{q} \right\}. \quad (3.6)$$

where $\mathbf{q}$ is a vector of predefined quantile levels. In particular, given a number of quantiles $n_{\text{quantiles}}$, one can consider the quantile level vector

$$\mathbf{q} = \left(\frac{v}{n_{\text{quantiles}} + 1} \right)_{v=1, \dots, n_{\text{quantiles}}}.$$

This approach is equivalent to pre-binarizing and using the boolean-featured model.

To simplify interpretability of and control over the model, we define rule complexity as

$$c_k = 1, \quad k \in \mathcal{K}.$$

This implies that C is the maximum cardinality of a ruleset.

3.2 Implementation

The threshold model as described in Section 3.1 is a special case of the ramp loss model. We refer the reader to Chapter 4 for details.

3.3 Methodology

First, we focus on simple rulesets for low PD-L1 expression thresholds. High values of Q_y led to very little learning as discussed in Section 2.5. Therefore, we restrict ourselves to $Q_y = 0.5, 0.6$. We are interested in the performance of very simple rulesets and what they look like. Therefore, we study

$$C = 1, 2 \quad \text{and} \quad D = 1, 2, 3,$$

training on Broad in full. We fix $n_{\text{quantiles}} = 9$ and consider both lower-bounds and upper-bounds. The use of the exact pricer has proven to be time-consuming, especially for higher $n_{\text{quantiles}}$. We are interested in high $n_{\text{quantiles}}$ to rule out the effect of course discretization on ruleset performance. To avoid wasting time and compute (that could otherwise be used by other users of the HPC), we use only the heuristic pricer with beam widths (1000, 1000) for $D = 2$ and (1000, 1000, 1000) for $D = 3$. While this removes optimality guarantees like those discussed in Section D.1, rulesets still provide a proof that a certain performance can be attained. Furthermore, optimal solutions for small C and small D will be discussed in Section 3.5.

The total time limit is set to 3 hours, which is much more than needed for this setup.

Next, we study the effect of increasing C and D on performance. Again, Broad is used in full for training. The settings for this experiment are analogous to the experiment above.

Finally, we study the generalization of simple rulesets for $Q_y = 0.5, 0.6$. We apply 10-fold stratified cross-validation on Broad. Then we compare train and test accuracy. We set $D = 2$ and study $C = 1, 2, 3$. We also study the reliability of these solutions by counting how often each ruleset occurs among splits. In this count, we disregard the threshold value and direction (lower-bound or upper-bound) of each clause, but identify a ruleset only by which TFs appear together in each rule. Ideally, the ruleset found is consistent, as it is meant to be a general rule governing PD-L1 expression.

3.4 Results

The results for $C = 1$ are summarized in Table 3.1. We report accuracy (Acc.), precision (Prec.) and recall (Rec.). A threshold is shown as both absolute value (i.e. -1.26) and quantile level (i.e. $Q(0.20)$) with respect to the observed activity scores of this TF in the training dataset. With merely one rule, a remarkable accuracy can be achieved. Increasing D consistently yields the same rule, plus a new clause. Furthermore, the rulesets found for $Q_y = 0.5$ and $Q_y = 0.6$ are distinct. There is a performance just going from $D = 1$ to $D = 2$, but no such jump going from $D = 2$ to $D = 3$. This suggests considering a compound rule, rather than simply a clause, has some effect, but adding more clauses in the compound rule does not have much effect.

The results for $C = 2$ are summarized in Table 3.2. By allowing for two rules, accuracy only increases marginally. Increasing D from 2 to 3 again yields practically no performance gain. Notably, increasing D now yields qualitatively different rulesets. Another observation is that for $Q_y = 0.5$, $C = 2$, $D = 2$, both rules involve the same TF.

The results for the second experiment are shown in Figure 3.1. First observe that setting $C = 5$, $D = 3$ in the threshold model is comparable to setting $C = 20$, $D = 3$ in the boolean model. Therefore, compared to the experiment described in Section 2.4, this experiment considers lower ruleset complexity. As seen in previous tables, low values of C yield reasonable performance. While accuracy increases with C , the increase is marginal considering the increase in perceived ruleset complexity. To clarify, a table like Table 3.1 for $C = 5$, $D = 3$ would span several pages. Increasing D has little effect on performance and even $D = 1$ shows reasonable performance for low Q_y . However, as seen in the previous experiment, there is some performance jump

Table 3.1: Results for the full Broad dataset for $C = 1$ and small rules ($D = 1, 2, 3$).

Q_y	C	D	Acc.	Prec.	Rec.	Ruleset
0.5	1	1	0.69	0.69	0.69	$\{\text{STAT2} \geq -0.20 Q(0.50)\}$
		2	0.71	0.72	0.68	$\{\text{FOS} \geq -1.26 Q(0.20) \wedge \text{STAT2} \geq -0.20 Q(0.50)\}$
		3	0.71	0.73	0.68	$\{\text{FOS} \geq -1.26 Q(0.20) \wedge \text{IRF1} \geq -1.59 Q(0.10) \wedge \text{STAT2} \geq -0.20 Q(0.50)\}$
0.6	1	1	0.70	0.67	0.50	$\{\text{FOSL1} \geq 0.41 Q(0.70)\}$
		2	0.74	0.70	0.59	$\{\text{FOSL1} \geq 0.13 Q(0.60) \wedge \text{STAT2} \geq -0.74 Q(0.30)\}$
		3	0.74	0.72	0.59	$\{\text{FOS} \geq -0.78 Q(0.30) \wedge \text{FOSL1} \geq 0.13 Q(0.60) \wedge \text{STAT2} \geq -0.74 Q(0.30)\}$

going from $D = 1$ to $D = 2$.

The cross-validation results for $Q_y = 0.5, 0.6$ and $D = 2$ are shown in Figure 3.2. If $C = 1$, then train and test accuracy are close. Increasing C marginally improves training accuracy, but does not significantly improve test accuracy. Hence, it appears that increasing C causes overfitting rather than learning.

At this point, it appears that the most effective rulesets are the absolute simplest ones: those with one rule and only two TF. It may be that more complicated rulesets simply do not perform better on this dataset. It may also be that the model does not behave as intended. Hence, we study an alternative approach with an independent Python implementation in Section 3.5.

Finally, the ruleset frequency counts are given in Table 3.3. This reveals that rulesets are sometimes (nearly) consistent, but not always. A possible explanation for the inconsistency is that some TF are known to bind, for instance RELA and NFKB1. It is therefore possible that the complex claimed to be promoting PD-L1 expression is a complex of RELA, NFKB1 and STAT2. However, NFKB1 only appears once in the entire table. More generally, the high cross-correlation between TF activity scores as observed in Figure A.1a suggests that TF could be effectively interchangeable in the dataset. Whether this is a real-life phenomenon or an artifact of the TF estimation is unclear.

Table 3.2: Results for the full Broad dataset for $C = 2$ and small rules ($D = 1, 2, 3$).

Q_y	C	D	Acc.	Prec.	Rec.	Ruleset
0.5	2	1	0.69	0.69	0.69	$\{\text{STAT2} \geq -0.20 \text{ Q}(0.50)\}$
		2	0.73	0.78	0.64	$\{\text{FOSL1} \geq 0.13 \text{ Q}(0.60) \wedge \text{STAT2} \geq -0.74 \text{ Q}(0.30)\}$
		3	0.73	0.78	0.65	$\{\text{STAT2} \geq 1.05 \text{ Q}(0.80) \wedge \text{FOSL1} \leq 0.13 \text{ Q}(0.60) \wedge \text{FOS} \geq -0.78 \text{ Q}(0.30) \wedge \text{RELA} \geq 0.03 \text{ Q}(0.50) \wedge \text{STAT2} \geq -0.20 \text{ Q}(0.50)\}$
0.6	2	1	0.72	0.68	0.58	$\{\text{NFKB1} \geq -1.17 \text{ Q}(0.20) \wedge \text{MYC} \leq -3.20 \text{ Q}(0.70) \wedge \text{TBX21} \leq -0.46 \text{ Q}(0.20)\}$
		2	0.75	0.70	0.65	$\{\text{FOSL1} \geq 0.41 \text{ Q}(0.70)\}$
		3	0.76	0.73	0.66	$\{\text{STAT2} \geq 1.95 \text{ Q}(0.90) \wedge \text{FOSL1} \geq 0.13 \text{ Q}(0.60) \wedge \text{STAT2} \geq -0.74 \text{ Q}(0.30)\}$
						$\{\text{IRF1} \geq 2.39 \text{ Q}(0.90) \wedge \text{GATA2} \leq 0.92 \text{ Q}(0.70)\}$
						$\{\text{FOS} \geq -0.78 \text{ Q}(0.30) \wedge \text{FOSL1} \geq 0.13 \text{ Q}(0.60) \wedge \text{STAT2} \geq -0.74 \text{ Q}(0.30)\}$
						$\{\text{STAT1} \geq 0.44 \text{ Q}(0.80) \wedge \text{STAT6} \geq -0.36 \text{ Q}(0.20) \wedge \text{GATA2} \leq 0.92 \text{ Q}(0.70)\}$

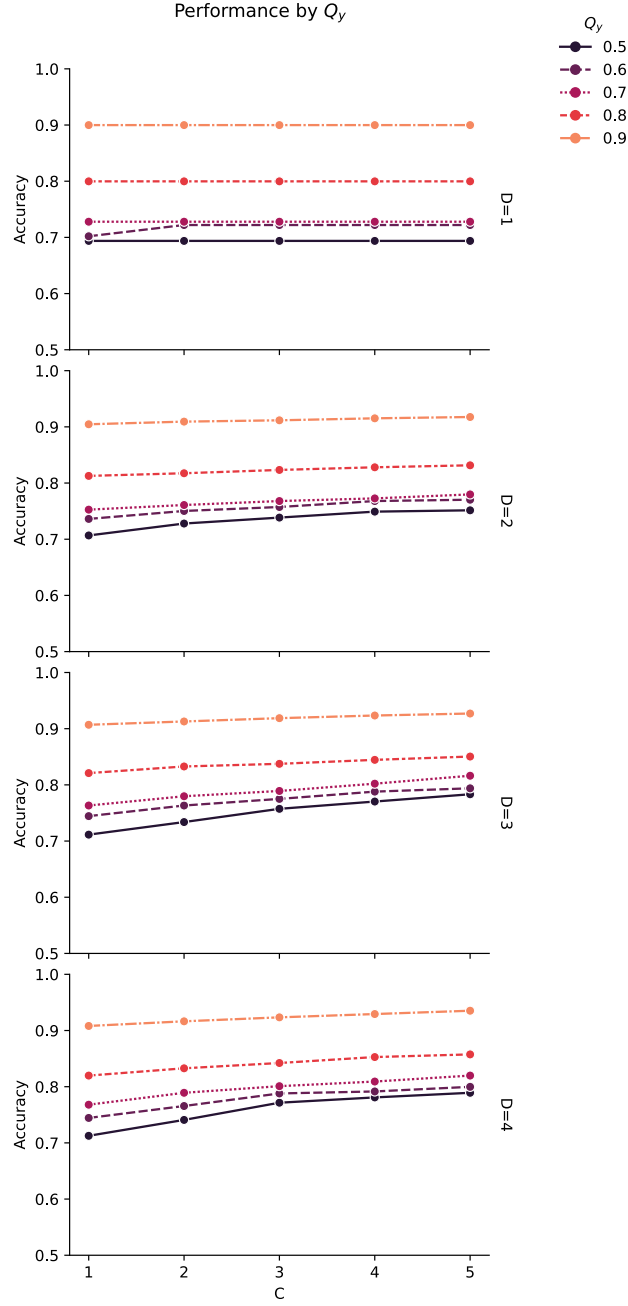


Figure 3.1: Accuracy of results for the full Broad dataset for $C = 1, 2, 3, 4, 5$, $D = 1, 2, 3, 4$ and $Q_y = 0.5, 0.6, 0.7, 0.8, 0.9$.

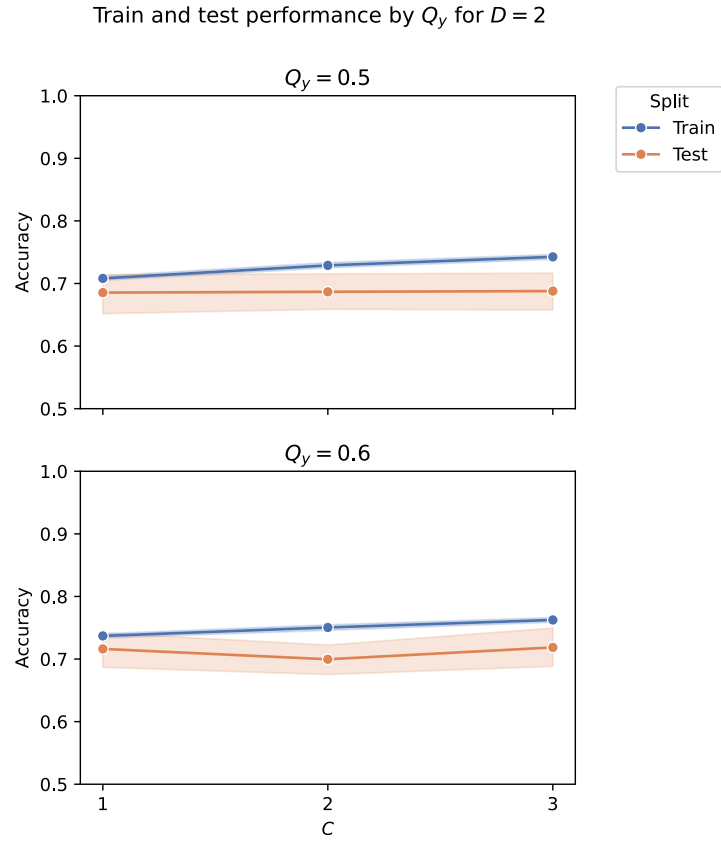


Figure 3.2: Confidence intervals (95%) for train and test accuracy of results under 10-fold stratified cross-validation on the Broad dataset, $C = 1, 2, 3$.

Table 3.3: Ruleset frequency count for the results of 10-fold stratified cross validation on the Broad dataset. In the frequency count, the threshold values and directions have been disregarded. Therefore, this table shows how often distinct TF combinations appear when only one rule is to be deduced from a cross-validation split of the Broad dataset

Q_y	C	D	TF combination	Frequency
0.5	1	2	RELA $\wedge$ STAT2	6
			FOS $\wedge$ STAT2	3
			NFKB1 $\wedge$ STAT2	1
		3	FOS $\wedge$ IRF1 $\wedge$ STAT2	5
			FOS $\wedge$ RELA $\wedge$ STAT2	3
			IRF1 $\wedge$ RELA $\wedge$ STAT2	2
0.6	1	2	FOSL1 $\wedge$ STAT2	10
		3	FOS $\wedge$ FOSL1 $\wedge$ STAT2	9
			FOSL1 $\wedge$ STAT2 $\wedge$ STAT6	1

3.5 Greedy algorithm

As an additional approach we explore a greedy algorithm with two goals in mind: to investigate whether this simpler approach performs similarly, and to verify that optimal rulesets for simple cases (low C and D) are similar to those produced by our earlier approach.

In the greedy approach, we first find through brute force complete enumeration a rule with minimum 0-1 loss. Then, the positively classified points are removed from the dataset and the cycle repeats. Note that this greedy method is essentially a strategy to find a rule list rather than a rule set. For $C = 1$, the greedy approach finds an optimal ruleset.

Mathematically, we consider in step τ the points $\mathcal{I}_\tau$ and the corresponding positive points $\mathcal{P}_\tau$ and negative points $\mathcal{N}_\tau$. Denote by rule k_τ the rule found in step τ . Then $\mathcal{I}_0 = \mathcal{I}$ and

$$\mathcal{I}_{\tau+1} = \mathcal{I}_\tau \setminus \{i \in \mathcal{I}_\tau : k_\tau \in \mathcal{K}_i\} = \mathcal{I}_\tau \setminus \mathcal{I}_{\{k_\tau\}}, \quad \tau \in \mathbb{N}_+.$$

Equivalently,

$$\mathcal{I}_{\tau+1} = \{i \in \mathcal{I}_\tau : y_{\{k_\tau\}}(\mathbf{X}_i) \neq 1\}, \quad \tau \in \mathbb{N}_+.$$

Algorithm 1 Greedy Ruleset Selection

Require: $\mathcal{I}, \mathbf{X}, \mathbf{y}, C, D$

- 1: $\mathcal{I}_1 \leftarrow \mathcal{I}$
 - 2: **for** $\tau = 1, \dots, C$ **do**
 - 3: Find rule k_τ with at most D clauses minimizing 0-1 loss w.r.t. $\mathbf{X}$ and $\mathbf{y}$
 - 4: $\mathcal{I}_{\tau+1} \leftarrow \{i \in \mathcal{I}_\tau : \hat{y}_{\{k_\tau\}}(\mathbf{X}_i) \neq 1\}$
 - 5: **end for**
 - 6: **return** $(k_\tau)_{\tau=1, \dots, C}$
-

An advantage of the greedy algorithm is that it is easy to understand and to implement. Furthermore, discretization via thresholding is done in every step separately. This means that new, more relevant threshold values may be found in later steps, when a part of the points is no longer considered. A disadvantage is that there is no optimality guarantee. Additionally, the Python implementation, while highly readable, is slow. This is in part due to enumeration of all rules in each step, which is slow for larger values of D (in our experiments, for $D \geq 3$). Another aspect is the high-level nature of Python, hurting performance, despite the use of NumPy where possible.

The results for $D = 1$ are summarized in Table 3.5. While the greedy method was configured to continue after step two, there did not exist a rule that improved performance. The first rule provides reasonable performance.

The second rule provides barely any improvement. For both $Q_y = 0.5$ and $Q_y = 0.6$, the same features appear in the rules.

The results for $D = 2$ are summarized in Table 3.5. In both tables, the value of adding another rule quickly diminishes. The second and third rules barely increase accuracy. The third rule found covers only a small fraction of the remaining positive points. Compared to Table 3.1 and Table 3.2, the accuracy observed is similar, albeit slightly lower. Qualitatively, there is some overlap in the rules found, but there are differences.

The previous method (Section 3.4) performs similar to the greedy method. Since for $C = 1$ greedy is optimal, this suggests that the previous method is working as intended. Sometimes, the greedy method outperforms the previous method. There are two contributing factors for this. First, the greedy method has more freedom in later steps. Second, the previous approach is a heuristic, as it uses price-then-branch, and it was configured to use only the heuristic pricer. Nevertheless, overall performance is similar.

The conclusion of the experiments in this chapter is two-fold.

- Compared to Section 2.5, decreasing complexity only moderately decreases performance. Instead, very simple rulesets can achieve a comparable accuracy, even when only using one clause per rule, which was not expected. It appears that the data is highly correlated and noisy, making it unsuitable for this off-the-shelf classifier model.
- The rulesets found are not consistent. The cross-validation experiment showed that rulesets vary among splits. The inconsistency of rules is supported by the wide variety of TF combinations observed in this chapter's tables.

We will therefore alter the model in an attempt to make it more robust and handle noise better, hopefully providing more meaningful rulesets.

Table 3.4: Rules found in each step of the greedy algorithm, for $Q_y = 0.5$ (top) and $Q_y = 0.6$ (bottom). The full Broad dataset is used for training and $D = 1$. In both cases, accuracy cannot be improved by any additional rule after step two.

τ	# in $\mathcal{I}_\tau$	# in $\mathcal{P}_\tau$	# covered by k_τ	# in $\mathcal{P}_\tau$ covered by k_τ	Acc.	Rule
1	849	425	425	295	0.69	$\{\text{STAT2} \geq -0.199 \text{ Q}(0.500)\}$
2	424	130	43	23	0.70	$\{\text{FOSL1} \geq 0.620 \text{ Q}(0.899)\}$

τ	# in $\mathcal{I}_\tau$	# in $\mathcal{P}_\tau$	# covered by k_τ	# in $\mathcal{P}_\tau$ covered by k_τ	Acc.	Rule
1	849	340	255	171	0.70	$\{\text{FOSL1} \geq 0.413 \text{ Q}(0.700)\}$
2	594	169	60	39	0.72	$\{\text{STAT2} \geq 1.256 \text{ Q}(0.899)\}$

Table 3.5: Rules found in each step of the greedy algorithm, for $Q_y = 0.5$ (top) and $Q_y = 0.6$ (bottom). The full Broad dataset is used for training and $D = 2$.

τ	# in $\mathcal{I}_\tau$	# in $\mathcal{P}_\tau$	# covered by k_τ	# in $\mathcal{P}_\tau$ covered by k_τ	Acc.	Rule
1	849	425	398	287	0.71	$\{\text{FOS} \geq -1.258 \text{ Q}(0.200) \wedge \text{STAT2} \geq -0.199 \text{ Q}(0.500)\}$
2	451	138	38	26	0.72	$\{\text{FOSL1} \geq 0.192 \text{ Q}(0.799) \wedge \text{STAT2} \geq -0.701 \text{ Q}(0.601)\}$
3	413	112	26	18	0.73	$\{\text{RELA} \geq -2.926 \text{ Q}(0.102) \wedge \text{TBX21} \leq -0.793 \text{ Q}(0.102)\}$

τ	# in $\mathcal{I}_\tau$	# in $\mathcal{P}_\tau$	# covered by k_τ	# in $\mathcal{P}_\tau$ covered by k_τ	Acc.	Rule
1	849	340	288	202	0.74	$\{\text{FOSL1} \geq 0.135 \text{ Q}(0.600) \wedge \text{STAT2} \geq -0.743 \text{ Q}(0.300)\}$
2	561	138	42	29	0.76	$\{\text{STAT2} \geq 1.085 \text{ Q}(0.899) \wedge \text{GATA2} \leq 1.156 \text{ Q}(0.799)\}$
3	519	109	12	11	0.77	$\{\text{NFKB1} \geq 0.012 \text{ Q}(0.599) \wedge \text{FOXO1} \leq -1.193 \text{ Q}(0.200)\}$

Chapter 4

Ramp loss model

In this chapter, we explore an alternative loss function, the ramp loss, inspired by the support vector machine (SVM). The threshold model discussed in Section 3 utilizes the so-called hard-margin loss (Figure 4.1c), also known as 0-1 loss. This means a correctly classified point induces a loss of 0, while an incorrectly classified point induces a loss of 1. It may be beneficial to assign a lower loss to misclassified points that are “close” to the rule set.

The SVM is a mathematical programming-based binary classification method developed by Vapnik in 1998 and Cortes and Vapnik in 1995 [25, 10, 8]. The classifier finds a hyperplane in the feature space that aims to separate positive datapoints from negative datapoints. The formulation proposed by Vapnik and co-authors uses the hinge loss, a continuous measure for misclassification. Specifically, the misclassification error is directly proportional to the distance between the point and the plane. The behavior of the hinge loss is illustrated in Figure 4.1a. The hinge loss makes the SVM sensitive to outliers. One method to increase robustness of the SVM, studied by several authors ([8, 7]), is the use of the ramp loss, illustrated in Figure 4.1b.

Besides being more robust than the hinge loss, the ramp loss has additional advantages. Points that are “close” to a rule induce a lower loss. This could reveal rules that would otherwise remain unchosen due to noise; if many positive and negative points live near the border of an otherwise good rule, the hard-margin loss can turn out large despite the rule being promising. This is especially true in the case of a small selection of thresholds (i.e., a limited number quantile levels) as opposed to full freedom in selecting thresholds. In the former case, it may be that an available threshold happens to fall on the “wrong side” of a relevant cloud of points. A ramp loss could ease this pain by reducing the loss of points that are close to this threshold. Additionally, the ramp

loss could prove more robust than 0-1 loss, as the change in loss as a result of a perturbation of the input is limited. This is due to the reduced “steepness” of the ramp loss as opposed to the hard-margin loss.

This chapter covers the modification of the threshold model from the hard-margin loss to the ramp loss, which is our contribution. The resulting ramp loss model is a generalization of the threshold model.

Two variants are considered: “additive” and “non-additive”. The latter is an attempt to overcome issues with the former. A mathematical description of the additive model is treated in Section 4.1 and Section 4.2. Examples of the model’s behavior on simple, artificial examples are covered in Section 4.3. These examples are in two dimensions and help develop a visual intuition. The subsequently developed non-additive variant is covered in Section 4.4. Implementation is discussed in Section 4.5. The experiments and their results are discussed in Section 4.6 and Section 4.7, respectively.

Readers who are mainly interested in the experiments and their results are advised to continue reading Section 4.3. This is the section with examples, which helps to build an intuition for the model. Section 4.4 is mathematical, but contains more examples worth reading.

4.1 Model description

We will now reformulate (2.6) to incorporate a multidimensional generalization of the ramp loss. To do this, first observe that the objective (2.6a) can be written as

$$\sum_{i \in \mathcal{P}} \zeta_i + \sum_{i \in \mathcal{N}} \sum_{k \in \mathcal{K}} \gamma_{ik} w_k,$$

where

$$\gamma_{ik} = \begin{cases} 1, & \text{if } k \in \mathcal{K}_i, \\ 0, & \text{otherwise.} \end{cases}$$

In γ_{ik} , we recognize a hard-margin loss. Specifically, for a negative point $i \in \mathcal{N}$, the value γ_{ik} is exactly the loss induced if k is used. Similarly, constraint (2.4b) can be written as

$$\zeta_i + \sum_{k \in \mathcal{K}} \gamma_{ik} w_k \geq 1, \quad \forall i \in \mathcal{P}.$$

If a positive point $i \in \mathcal{P}$ is uncovered, this implies the sum will be zero and hence $\zeta_i = 1$, which corresponds to an induced loss of 1. The value γ_{ik} represents a reduction of this loss if rule k is used. If k covers the point, this value is 1. Using rule k then allows ζ_i to be reduced to 0.

By defining γ suitably, we can introduce a multidimensional variant of the ramp loss. When doing so, the values of γ need no longer be binary, hence ζ

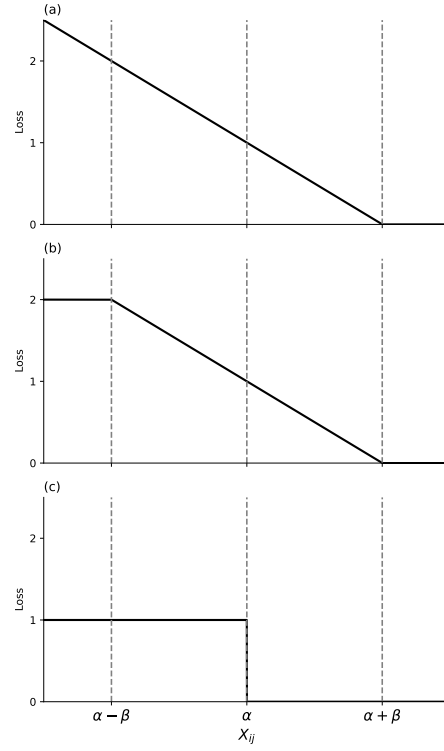


Figure 4.1: Loss for a positive datapoint i subject to rule $X_{.j} \geq \alpha$ plotted against the feature value X_{ij} with (a) the hinge loss, (b) the ramp loss and (c) the hard margin loss. The width of the margin is given by $\beta > 0$. An observation whose feature value X_{ij} falls between $\alpha - \beta$ and $\alpha + \beta$ lies in the margin.

should no longer be binary. Furthermore, the ramp loss of a point is anywhere between 0 and 2. The ramp loss model is therefore given by

$$\min \sum_{i \in \mathcal{P}} \zeta_i + \sum_{i \in \mathcal{N}} \sum_{k \in \mathcal{K}} \gamma_{ik} w_k \quad (4.1a)$$

$$\text{s.t. } \zeta_i + \sum_{k \in \mathcal{K}} \gamma_{ik} w_k \geq 2, \quad \forall i \in \mathcal{P} \quad (4.1b)$$

$$\sum_{k \in \mathcal{K}} c_k w_k \leq C, \quad (4.1c)$$

$$w \in \{0, 1\}^{\mathcal{K}}, \zeta \in [0, 2]^{\mathcal{P}}, \quad (4.1d)$$

To define γ , we first define a decision function $d : \mathcal{P}(\mathcal{L}) \rightarrow \mathbb{R}^{\mathcal{I}}$ such that $d_i(k)$ is non-negative if rule k covers point i and negative otherwise. Furthermore, the value $d_i(k)$ quantifies how far i is from the boundary of k . Define

$$d_i(k) = \min_{(j, \alpha) \in k} X_{ij} - \alpha, \quad i \in \mathcal{I}, k \in \mathcal{K} \setminus \{\emptyset\}. \quad (4.2)$$

If k covers i , then any element in the minimum is non-negative, meaning $d_i(k)$ is the distance to the closest threshold. If rule k does not cover i , then there exists some clause $(j, \alpha) \in k$ that cuts off i . Choose such (j, α) . It follows that $X_{ij} - \alpha < 0$ and so $d_i(k) < 0$. By extension, we have in this case that

$$d_i(k) = - \max_{(j, \alpha) \in k : X_{ij} < \alpha} \alpha - X_{ij}, \quad i \in \mathcal{I}.$$

This means that $d_i(k)$ is the (negated) distance to the furthest unsatisfied threshold. Therefore, the value $-d_i(k)$ can be interpreted as a distance to the covered region. To account for the empty rule, we set

$$d_i(\emptyset) = \infty, \quad i \in \mathcal{I}. \quad (4.3)$$

We define

$$\gamma_{ik} = \begin{cases} 0, & \text{if } d_i(k) \leq -\beta, \\ 1, + \frac{1}{\beta} d_i(k), & \text{if } d_i(k) \in (-\beta, \beta], \\ 2, & \text{if } d_i(k) > \beta, \end{cases} \quad i \in \mathcal{I}, k \in \mathcal{K}. \quad (4.4)$$

Explain further?

This definition realizes a generalized ramp loss in the master problem (4.1).

We refer to this model as the “additive” ramp loss model. This name is due to the additive nature of loss reduction for positive points. Recall (4.1b). Several rules, when used together, can all contribute to reducing a ζ -variable. For example, if point i lies in several margins, the corresponding loss ζ_i can still be 0, which is equivalent to being far inside the covered region of the ruleset.

As $\beta \rightarrow 0$, the ramp loss converges pointwise to the hard-margin loss (multiplied by 2) for all values of X_{ij} except the corresponding α . Assuming no

feature values lies on such threshold, it follows that the ramp loss model is equivalent to the threshold model when β is arbitrarily small. To see this, observe that for β small enough, γ and ζ are the coefficients used in the threshold model, multiplied by 2.

In a realistic setting, it is to be expected that few, or none, of the datapoints lie exactly on the threshold. There are two reasons for this. First, continuous measurements typically include a measurement error, making it less likely for several datapoints to have an equal feature value. Second, the linear interpolation used in the quantile function is likely to yield a non-observed value. Therefore, in a practical setting with continuous features, the ramp loss model with β arbitrarily small is practically equivalent to the threshold model. For convenience, we define for the special case $\beta = 0$,

$$\begin{aligned}\gamma_{ik} &= 2 \cdot \mathbb{I}(d_i(k) \geq 0) \\ &= \begin{cases} 0, & \text{if } \exists(j, \alpha) \in k : X_{ij} < \alpha, \\ 2, & \text{if } X_{ij} \geq \alpha \quad \forall(j, \alpha) \in k, \end{cases} \quad i \in \mathcal{I}, k \in \mathcal{K}. \end{aligned} \quad (4.5)$$

For this choice of γ , the model is exactly equivalent to the threshold model. We then use the implementation of the ramp loss model with $\beta = 0$ to solve the threshold model. The goal is now to express the pricing problem in terms of γ and study how this problem may be solved.

4.2 Pricing problem

The reformulations (4.1a) and (4.1b) affect the pricing problem. We will now derive the pricing problem, study its complexity and construct a mixed integer program (MIP) that can be used to solve it. Given non-negative dual values $(\mu, \lambda) \in \mathbb{R}_+^{\mathcal{P}} \times \mathbb{R}_+$, the reduced cost of a rule $k \in \mathcal{K}$ is given by

$$\rho_k = \sum_{i \in \mathcal{N}} \gamma_{ik} - \sum_{i \in \mathcal{P}} \mu_i \gamma_{ik} + \lambda c_k. \quad (4.6)$$

The derivation is discussed in Section B.2. For $\beta > 0$, we have that

$$\begin{aligned}\gamma_{ik} &= \left[1 + \frac{1}{\beta} d_i(k) \right]_0^2 \\ &= \left[1 + \frac{1}{\beta} \min_{(j, \alpha) \in k} X_{ij} - \alpha \right]_0^2 \\ &= \min_{(j, \alpha) \in k} \left[1 + \frac{1}{\beta} (X_{ij} - \alpha) \right]_0^2, \quad i \in \mathcal{I}, k \in \mathcal{K} \setminus \{\emptyset\}. \end{aligned} \quad (4.7)$$

where $[\cdot]_a^b$ clips $\cdot$ to $[a, b]$. Define

$$\gamma_{il} = \begin{cases} \left[1 + \frac{1}{\beta} (X_{ij} - \alpha) \right]_0^2, & \text{if } \beta > 0 \\ 2 \cdot \mathbb{I}(X_{ij} \geq \alpha), & \text{if } \beta = 0, \end{cases} \quad i \in \mathcal{I}, l \in \mathcal{L}. \quad (4.8)$$

Then

$$\gamma_{ik} = f_i(k) := \begin{cases} 2, & k = \emptyset, \\ \min_{l \in k} \gamma_{il}, & k \neq \emptyset, \end{cases} \quad i \in \mathcal{I}, k \in \mathcal{K}. \quad (4.9)$$

Minimizing (4.6) is equivalent to solving

$$\min_{S \subseteq \mathcal{L}} \sum_{i \in \mathcal{N}} f_i(S) - \sum_{i \in \mathcal{P}} \mu_i f_i(S) + \lambda c(S), \quad (4.10)$$

where $c : \mathcal{K} \rightarrow \mathbb{Z}_+$ is a function measuring rule complexity.

4.2.1 Complexity

In this section we discuss the complexity of the pricing problem (4.10) for various definitions of rule complexity. It turns out to be NP-hard in general. In Chapter 2, rule complexity for the boolean model was defined as

$$c(S) = 1 + |S|.$$

Theorem 4.2.1. [17] The problem (4.10) is NP-hard for $c(S) = 1 + |S|$.

Proof. The proof is analogous to the one given in [17] and is by reduction from the minimum vertex cover (MVC) problem, which is known to be NP-complete. Let $G = (V, E)$ be an undirected graph with at least one edge, otherwise the problem has a trivial solution. We construct an instance of (4.10) that finds a minimum cardinality vertex cover of G . Take $\mathcal{P} = \emptyset$. For each vertex $v \in V$, create a clause $l_v = (v, 1) \in \mathcal{L}$. For each edge $e = \{u, v\} \in E$, create a point $i_e \in \mathcal{N}$ such that

$$X_{i_e l_v} = \begin{cases} 0, & \text{if } v \in e, \\ 2, & \text{otherwise.} \end{cases}$$

Choose $\beta = 1$. It follows that

$$f_{i_e}(k) \in \{0, 2\}, \quad e \in E, k \in \mathcal{K}.$$

Precisely, we have that

$$f_{i_e}(S) = \begin{cases} 0, & \text{if } \exists l_v \in S : v \in e, \\ 2, & \text{otherwise,} \end{cases} \quad i_e \in \mathcal{N}, S \subseteq \mathcal{L}.$$

Additionally, let $\lambda = 1/(2 + |\mathcal{L}|)$. This leads to the problem

$$\min_{S \subseteq \mathcal{L}} \sum_{i \in \mathcal{N}} f_i(S) + \frac{1 + |S|}{2 + |V|}.$$

Observe that an optimal solution must have objective value strictly less than 1, as the solution containing every l_v , $v \in V$ has objective value strictly less than 1. Consequently, an optimal solution S must satisfy

$$\sum_{i \in \mathcal{N}} f_i(S) = 0 \implies \forall i_e \in \mathcal{N} \exists l_v \in S : v \in e,$$

which states exactly that S corresponds to a vertex cover. Hence, an optimal solution for this instance is a vertex cover and minimizes

$$\frac{1 + |S|}{2 + |V|},$$

which is equivalent to minimizing $|S|$. Therefore, this instance of the pricing problem, constructible in polynomial time, solves the MVC problem. $\square$

Analogously, the problem (4.10) is NP-hard for $c(S) = |S|$. This proof is not applicable for the case $c(S) = 1$, which was chosen in Chapter 3.

Theorem 4.2.2. The problem (4.10) is NP-hard for $c(S) = 1$.

Proof. The proof is again by reduction from the minimum vertex cover (MVC) problem and uses a similar construction. Let $G = (V, E)$ be an undirected graph with at least one edge. Recall the instance described in the proof of Theorem 4.2.1. Augment the instance by creating for each $v \in V$ a point $i_v \in \mathcal{P}$ with

$$X_{i_v l_{\bar{v}}} = \begin{cases} 0 & \text{if } v = \bar{v}, \\ 2 & \text{otherwise,} \end{cases} \quad v, \bar{v} \in V.$$

It follows that

$$f_{i_v}(S) = \begin{cases} 0 & \text{if } l_v \in S, \\ 2 & \text{otherwise,} \end{cases} \quad v \in V, S \subseteq \mathcal{L}.$$

Let $\lambda = 0$ and $\mu_i = 1/(1 + |V|)$, $i \in \mathcal{P}$. The problem is then given by

$$\min_{S \subseteq \mathcal{L}} \sum_{i \in \mathcal{N}} f_i(S) - \frac{1}{1 + |V|} \sum_{i \in \mathcal{P}} f_i(S).$$

Observe that the solution containing every l_v , $v \in V$ has objective value 0. Hence, an optimal solution has objective value at most 0. Note that

$$\frac{1}{1 + |V|} \sum_{i \in \mathcal{P}} f_i(S) \leq \frac{1}{1 + |V|} \sum_{i \in \mathcal{P}} 2 = \frac{2|V|}{1 + |V|} < 2,$$

To obtain an objective that is at most 0, it must then hold that

$$\sum_{i \in \mathcal{N}} f_i(S) < 2.$$

Let S be an optimal solution. Since $f_i(S) \in \{0, 2\}$ for all i , it follows that

$$\sum_{i \in \mathcal{N}} f_i(S) = 0,$$

This again implies that S corresponds to a vertex cover. Furthermore, solution S minimizes

$$\begin{aligned} -\frac{1}{1+|V|} \sum_{i \in \mathcal{P}} f_i(S) &= -\frac{1}{1+|V|} \sum_{i_v \in \mathcal{P}} \mathbb{I}(l_v \notin S) \\ &= -\frac{|V| - |S|}{1+|V|} \\ &= -\frac{|V|}{1+|V|} + \frac{|S|}{1+|V|}. \end{aligned}$$

which is equivalent to minimizing $|S|$. Therefore, this instance of the pricing problem, constructible in polynomial time, solves the MVC problem. $\square$

4.2.2 Mixed Integer Programming formulations

In this section, we explore MIP formulations of the pricing problem (4.10) and discuss challenges in solving them. Recall the reduced cost (4.6). Let $z \in \{0,1\}^{\mathcal{L}}$ again be integer variables representing which clauses are used in the rule. Consider the MIP

$$\min \quad \sum_{i \in \mathcal{N}} \gamma_i - \sum_{i \in \mathcal{P}} \mu_i \gamma_i + \lambda(z) \quad (4.11a)$$

$$\text{s.t.} \quad \gamma_i \leq 2 - (2 - \gamma_{il})z_l, \quad l \in \mathcal{L}, i \in \mathcal{P}, \quad (4.11b)$$

$$\gamma_i \geq \gamma_{il} - \sum_{l' \in \mathcal{L} : \gamma_{il'} < \gamma_{il}} (\gamma_{il} - \gamma_{il'})z_{l'}, \quad l \in \mathcal{L}, i \in \mathcal{N}, \quad (4.11c)$$

$$\sum_{l \in \mathcal{L}} z_l \leq D, \quad (4.11d)$$

$$z \in \{0,1\}^{\mathcal{L}}, \gamma \in [0,2]^{\mathcal{I}}. \quad (4.11e)$$

Denote by $\bar{\gamma}_{\text{OPT}}, \bar{z}_{\text{OPT}}$ an optimal solution for this IP. Recall that $\bar{z}_{\text{OPT}}$ is binary. Define

$$L_{\text{OPT}} = \{l \in \mathcal{L} : \bar{z}_{\text{OPT},l} = 1\}.$$

Claim 4.2.1.

$$\bar{\gamma}_{\text{OPT},i} = f_i(L_{\text{OPT}}), \quad i \in \mathcal{P}.$$

Proof. Let $i \in \mathcal{P}$ and $l \in \mathcal{L}$. If $\bar{z}_{\text{OPT},l} = 0$, the corresponding constraint (4.11b) becomes redundant. If $\bar{z}_{\text{OPT},l} = 1$, the constraint becomes $\gamma_i \leq \gamma_{il}$. Therefore, if $L_{\text{OPT}} \neq \emptyset$ it holds that

$$\gamma_i \leq \min_{l \in L_{\text{OPT}}} \gamma_{il}.$$

If $L_{\text{OPT}} = \emptyset$, we have that $\gamma_i \leq 2$ by the variable bound. In any case,

$$\gamma_i \leq f_i(L_{\text{OPT}}).$$

There are no other constraints involving γ_i . Since γ_i has a negative coefficient in the objective, this inequality will be an equality in an optimal solution. $\square$

Claim 4.2.2.

$$\bar{\gamma}_{\text{OPT},i} = \min_{l \in L_{\text{OPT}}} \gamma_{il}, \quad i \in \mathcal{N}.$$

Proof. Let $i \in \mathcal{N}$. If $L_{\text{OPT}} = \emptyset$, the only lower-bound involving γ_i is the variable bound, so $\gamma_i \geq 0$.

Assume $L_{\text{OPT}} \neq \emptyset$. Let $l^* \in L_{\text{OPT}}$ such that $\gamma_{il^*} = \min_{l \in L_{\text{OPT}}} \gamma_{il}$. For this i and l , the constraint (4.11c) is

$$\gamma_i \geq \gamma_{il}.$$

Now let $l \in \mathcal{L}$. We consider the right hand side of (4.11c). We have that

$$\gamma_{il} - \sum_{l' \in \mathcal{L} : \gamma_{il'} < \gamma_{il}} (\gamma_{il} - \gamma_{il'}) z_{l'} = \gamma_{il} - \sum_{l' \in L_{\text{OPT}} : \gamma_{il'} < \gamma_{il}} (\gamma_{il} - \gamma_{il'}).$$

If the sum is empty, then it must hold that $\gamma_{il} = \gamma_{il^*}$ and the right hand side equals γ_{il^*} . If the sum is non-empty, we have

$$\begin{aligned} \gamma_{il} - \sum_{l' \in L_{\text{OPT}} : \gamma_{il'} < \gamma_{il}} (\gamma_{il} - \gamma_{il'}) &\leq \gamma_{il} - \max_{l' \in L_{\text{OPT}} : \gamma_{il'} < \gamma_{il}} (\gamma_{il} - \gamma_{il'}) \\ &= \min_{l' \in L_{\text{OPT}} : \gamma_{il'} < \gamma_{il}} \gamma_{il'} \\ &= \gamma_{il^*}. \end{aligned}$$

Hence, $\gamma_i \geq \gamma_{il^*} = \min_{l \in L_{\text{OPT}}} \gamma_{il}$ is the tightest constraint on γ_i , induced by l^* .

Combining the case $L_{\text{OPT}} = \emptyset$ and $L_{\text{OPT}} \neq \emptyset$, we get that

$$y_i \geq f_i(L_{\text{OPT}}),$$

is the tightest lower-bound on γ_i . Since γ_i has a positive coefficient in the objective, this inequality will be an equality for an optimal solution. From this, the result follows. $\square$

We also impose the one-clause-per-feature constraint as given in (3.5).

The LP relaxation of (4.11) is not tight. The LP relaxation allows for *spreading*, by which several entries of z admit a fractional value.

Example 4.2.1 (Spreading). Let

$$\mathcal{N} = \{1, 3\}, \quad \mathcal{P} = \{2\}, \quad \mathcal{L} = \{a, b, f\}.$$

Consider the two-dimensional feature space and lower-bounds in Figure 4.2. The matrix (γ_{il}) is given by

	a	b	f
1	0	0	1
2	1/2	1/2	1
3	1/4	1/4	1

Revise these examples, specifically allowing for empty rules!

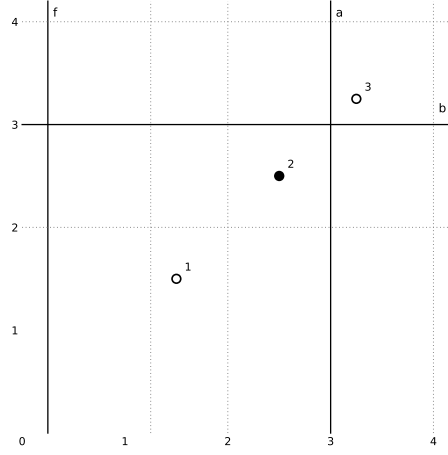


Figure 4.2: An example problem with a two-dimensional feature space, composed of one positive datapoint $X_2 = (2\frac{1}{2}, 2\frac{1}{2})$, two negative datapoints $X_1 = (1\frac{1}{2}, 1\frac{1}{2})$, $X_3 = (3\frac{1}{4}, 3\frac{1}{4})$ and clauses a, b, f representing $X_1 \geq 3, X_2 \geq 3, X_1 \geq \frac{1}{4}$, respectively. The margin width $\beta = 1$. The boundary of the margin is indicated with dotted lines.

Choose $D = 3$. In the initial pricing round, $\mu_i = 1$ and $\lambda = 0$. The LP relaxation of (4.11) attains an optimal objective value of $-1/2$, by

$$z = (\frac{1}{2}, \frac{1}{2}, 0), \quad \gamma_1 = 0, \quad \gamma_2 = \frac{3}{4}, \quad \gamma_3 = \frac{1}{4}.$$

Given z , the values of γ can be computed directly. The optimal objective value of the IP is $-1/4$, attained by several solutions, examples being

$$z = (1, 0, 0), \quad z = (0, 1, 0), \quad z = (1, 1, 0).$$

For any of these solutions, it holds that $\gamma_2 \leq 1/2$ because at least one of a or b is used. Meanwhile, we have $\gamma_2 = \frac{3}{4}$ in an optimal LP solution, like the one shown above. We refer to this as spreading, meaning the use of fractional values in z to admit larger γ_i for $i \in \mathcal{P}$, leading to a lower objective value.

In this example, the issue is resolved when branching on z_a or z_b . Under any of the branching decisions

$$z_a = 0, \quad z_a = 1, \quad z_b = 0, \quad \text{or} \quad z_b = 1,$$

the resulting LP solution is integer and $\gamma_2 = 1/2$.

4.2.3 Cutting planes

As we have seen, the relaxation of (4.11) is not tight. In practice, solving the MIP in this form is slow. To tighten the formulation, we study cutting planes

for (4.11b). Next, we will derive separation algorithms for both these cutting planes and the constraints (4.11c).

When considering only the constraints (4.11b), the problem is a special case of the Mixed Vertex Packing Problem. From this, it follows that for a given point i , the relevant cutting planes are

$$\begin{aligned} \gamma &\leq 2 - \sum_{l \in T} \bar{\gamma}_{il} z_l, \\ T &= \{l_1, l_2, \dots, l_t\} \subseteq \mathcal{L} \text{ such that } \gamma_{l_{j-1}} > \gamma_{l_j} \text{ for } j = 2, \dots, t, \end{aligned} \quad (4.12)$$

where

$$\begin{aligned} \bar{\gamma}_{il_1} &= 2 - \gamma_{il_1}, \\ \bar{\gamma}_{il_j} &= 2 - \gamma_{il_j} - (2 - \gamma_{il_{j-1}}) = \gamma_{il_{j-1}} - \gamma_{il_j}, \quad j = 2, \dots, t. \end{aligned}$$

For more details we refer to Section C.1. Another way to derive these cutting planes is via Extended Polymatroid Inequalities (EPIs). This is discussed in detail in Section C.2, where the simplified case of one datapoint is treated.

The idea is now to solve (4.11) by adding the cutting planes (4.12) and the constraints (4.11c) when necessary. To do this, we need an efficient separation algorithm.

4.2.4 Separation

We now describe a polynomial-time separation algorithm for (4.12) and then for (4.11c).

Separation of (4.12) for fixed i can be done in $\mathcal{O}(m \log m)$ time using a greedy algorithm. This may be shown in the context of polymatroids and EPI's [15], but this requires various mathematical concepts. Instead, we provide a standalone proof that is also the basis of the separation algorithm. Without loss of generality, consider a fixed point i and

$$2 \geq \gamma_1 \geq \gamma_2 \geq \dots \geq \gamma_m \geq 0,$$

each corresponding to a clause. Assume the z -variables are ordered accordingly. Fix $k \in [n]$ and let (z, γ) be a solution to the (partial) LP. Observe that a violated constraint exists if and only if there exist

$$\tau_1 < \tau_2 < \dots < \tau_t \in [n].$$

such that

$$\gamma > 2 - \sum_{\kappa=1}^t \bar{\gamma}_{\tau_\kappa} z_{\tau_\kappa}, \quad (4.13)$$

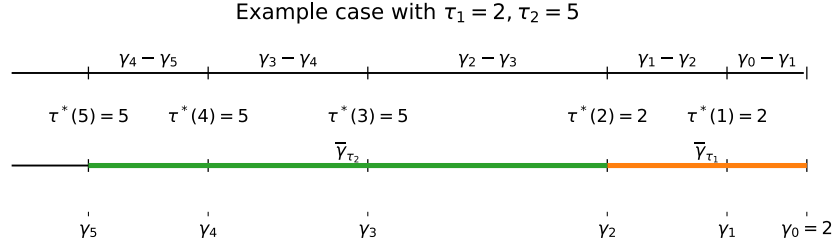


Figure 4.3: Example depicting the variables involved in the proof of the greedy separation algorithm for (4.12).

If the inequality holds for a minimizer of the right hand side, then we have found a new constraint. If the inequality does not hold for this minimizer, we may conclude that no violated constraint exists. Denote $\gamma_0 = 2, \tau_0 = 0$ for convenience. Note that

$$\bar{\gamma}_{\tau_k} = \gamma_{\tau_{k-1}} - \gamma_{\tau_k} = \left(\sum_{\tau=\tau_{k-1}+1}^{\tau_k} \gamma_{\tau-1} - \gamma_{\tau} \right), \quad \kappa = 1, \dots, t.$$

So

$$\begin{aligned} \sum_{\kappa=1}^t \bar{\gamma}_{\tau_k} z_{\tau_k} &= \sum_{\kappa=1}^t \left(\sum_{\tau=\tau_{k-1}+1}^{\tau_k} (\gamma_{\tau-1} - \gamma_{\tau}) \right) z_{\tau_k} \\ &= \sum_{\tau=1}^{\tau_t} (\gamma_{\tau-1} - \gamma_{\tau}) z_{\tau^*(\tau)}, \end{aligned}$$

where

$$\tau^*(\tau) = \min \{ \hat{\tau} \in \{\tau_1, \tau_2, \dots, \tau_t\} : \hat{\tau} \geq \tau \}, \quad \tau = 1, \dots, \tau_t,$$

so τ^* is effectively the next τ_k in the chosen subset of $[n]$ at index at least τ . A visualization of an example is given in Figure 4.3. To minimize the right hand side of (4.13), the sum needs to be maximized. Clearly, this is achieved when

$$z_{\tau^*(\tau)} = \max \{ z_{\hat{\tau}} : \hat{\tau} \geq \tau \}, \quad \tau = 1, \dots, t.$$

To achieve this, we can use a greedy algorithm. Precisely, find a $\hat{\tau}$ such that $z_{\hat{\tau}}$ is as large as possible. Set $\tau_1 = \hat{\tau}$. Then $\tau^*(\tau) = \hat{\tau}$ for all $\tau = 1, \dots, \hat{\tau}$, but $\tau^*(\tau)$ can never be $\hat{\tau}$ for larger τ by definition. Then find a $\hat{\tau} > \tau_1$ such that $z_{\hat{\tau}}$ is as large as possible and repeat. Do this until all $\tau \in [n]$ are ‘covered’ by some τ_k . This yields a maximizer of the sum in linear time, after sorting the z which takes $\mathcal{O}(m \log m)$ time.

Next, consider

$$0 \leq \gamma_1 \geq \gamma_2 \geq \dots \geq \gamma_m \leq 2,$$

and the constraint family

$$\gamma \geq \gamma_i - \sum_{i'=1}^i (\gamma_i - \gamma_{i'}) z_{i'}, \quad i \in [m], \quad (4.14)$$

which translates directly to (4.11c). A violated inequality exists if and only if

$$\gamma < \gamma_i - \sum_{i'=1}^i (\gamma_i - \gamma_{i'}) z_{i'}.$$

for some $i \in [m]$. Therefore, we are interested in a maximizer of the right hand side. Define $g : [m] \rightarrow \mathbb{R}$ by

$$g(i) = \gamma_i - \sum_{i'=1}^i (\gamma_i - \gamma_{i'}) z_{i'}.$$

Let $i \in [m-1]$. Then

check carefully

$$\begin{aligned} g(i+1) - g(i) &= \gamma_{i+1} - \sum_{i'=1}^{i+1} (\gamma_{i+1} - \gamma_{i'}) z_{i'} - \gamma_i + \sum_{i'=1}^i (\gamma_i - \gamma_{i'}) z_{i'} \\ &= \gamma_{i+1} - \sum_{i'=1}^i (\gamma_{i+1} - \gamma_{i'}) z_{i'} - \gamma_i + \sum_{i'=1}^i (\gamma_i - \gamma_{i'}) z_{i'} \\ &\quad + (\gamma_{i+1} - \gamma_{i+1}) z_{i+1} \\ &= \gamma_{i+1} - \gamma_i - \sum_{i'=1}^i (\gamma_{i+1} - \gamma_{i'} - \gamma_i + \gamma_{i'}) z_{i'} \\ &= \gamma_{i+1} - \gamma_i - \sum_{i'=1}^i (\gamma_{i+1} - \gamma_i) z_{i'} \\ &= (\gamma_{i+1} - \gamma_i) \left(1 - \sum_{i'=1}^i z_{i'} \right). \end{aligned}$$

Since $\gamma_{i+1} \geq \gamma_i$, it follows that

$$g(i+1) \geq g(i) \iff \sum_{i'=1}^i (\gamma_{i+1} - \gamma_i) z_{i'} \leq 1.$$

Hence, if $i \in [m-1]$ is the largest index such that

$$\sum_{i'=1}^i (\gamma_{i+1} - \gamma_i) z_{i'} \leq 1,$$

then $i+1$ is a maximizer for g . Denote this index by i^* . If $g(i^*) > \gamma$, then the corresponding constraint (4.14) is violated. Otherwise, no violated inequality exists. Finding such i^* can be done in linear time.

Verify implementation!
Do we actually do this?

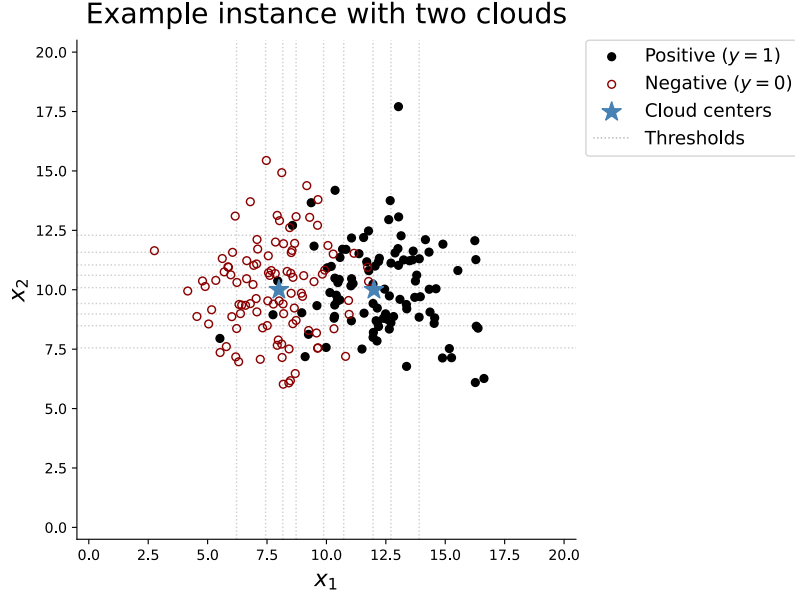


Figure 4.4: Instance of Example 4.3.1. The example consists of a negative and a positive cloud of 100 points each, following $\mathcal{N}((8, 10), 2)$ and $\mathcal{N}((12, 10), 2)$ respectively.

4.3 Examples

In this section, we study the behavior of the ramp loss model on some simple, artificial data. These examples employ a “full master problem”, meaning all relevant rules (columns) are considered. Precisely, given a positive integer D , all possible rules with at most D clauses satisfying the following conditions are considered:

- A feature appears at most once in any rule. So a feature appears in at most one clause within a rule.
- A feature and their negated version do not appear together in any rule. This means a feature cannot appear in both a “ $\geq$ ”-clause and a “ $\leq$ ”-clause within a rule.

The examples are in two dimensions. Recall that for $\beta = 0$, the ramp loss model is equivalent to the threshold model.

Example 4.3.1 (Two clouds). We construct two clouds of points according to multivariate normals. Precisely,

- A cloud of 100 negative points drawn from $\text{Normal}((8, 10), 2)$,

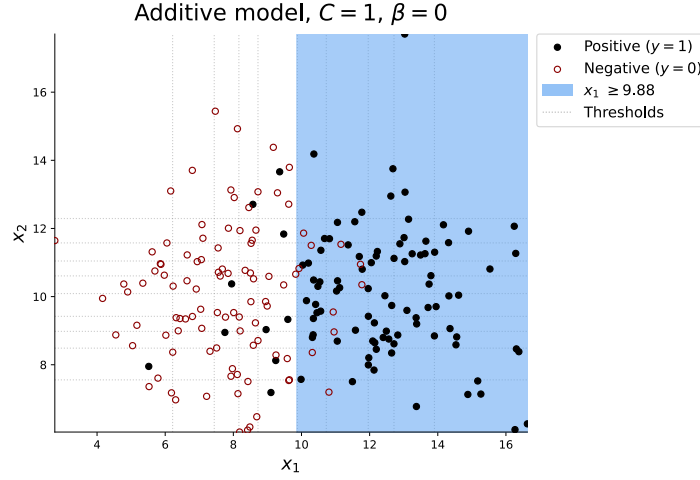


Figure 4.5: Solution to an instance of Example 4.3.1 computed with the ramp loss model, configured with $C = 1$, $\beta = 0$, $D = 2$.

- A cloud of 100 positive points drawn from $\text{Normal}((12, 10), 2)$.

Hence, the cloud centers share x_2 but differ in x_1 . An instance is depicted in Figure 4.4. Choosing $c = 1$ as rule complexity, we solve the ramp loss model for the parameters

$$C \in \{1, 2\}, \quad \beta \in \{0, 0.5, 1, 2, 3, 4, 5\}, \quad D = \{2\}.$$

For $\beta = 0$, the solution contains only the rule $x_1 \geq 9.88$ regardless of C . So, for this example, an increase in C does not cause the model to behave differently. This solution is depicted in Figure 4.5.

For $C = 1$ fixed, the same solution is found for β up until and including $\beta = 4$. For $C = 1$, $\beta = 5$, the rule is $x_1 \geq 10.73$ instead. We conclude that for $C = 1$, the ramp loss model finds a sensible solution for any of the tested β .

For $C > 1$, $\beta > 0$, the model behaves peculiarly: all solutions consist of two rules. For $C = 2$, $\beta = 0.5$ the ruleset consists of two adjacent, non-overlapping rules, as depicted in Figure 4.6a. The same holds for $C = 2$, $\beta = 1$, as shown in Figure 4.6b. These rulesets achieve a much lower objective than the rule $x_1 \geq 9.88$ found earlier. For $\beta = 0.5$, this can be partly attributed to the exclusion of the lower left corner in the covered region. More interestingly, for $\beta = 1$, the choice of this ruleset can be attributed to the additive nature of the model. To see this, recall that points near a margin induce a loss, even if the point resides on the desired side of the covered region. Looking closer, con-

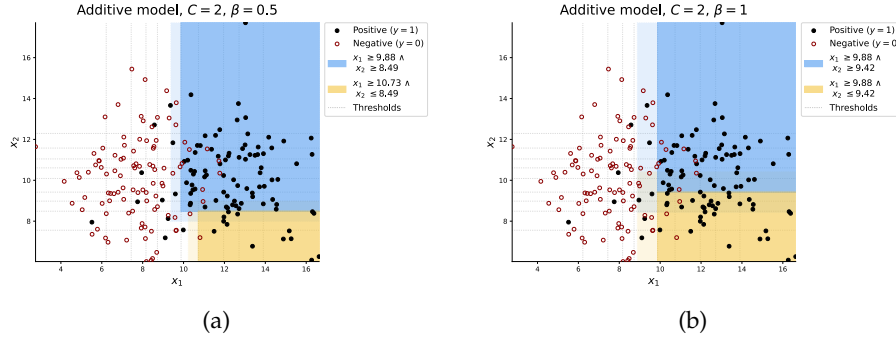


Figure 4.6: Solutions to an instance of Example 4.3.1 computed with the ramp loss model, configured with $C = 2, \beta = 0.5$ (a) and $C = 2, \beta = 1$ (b), respectively.

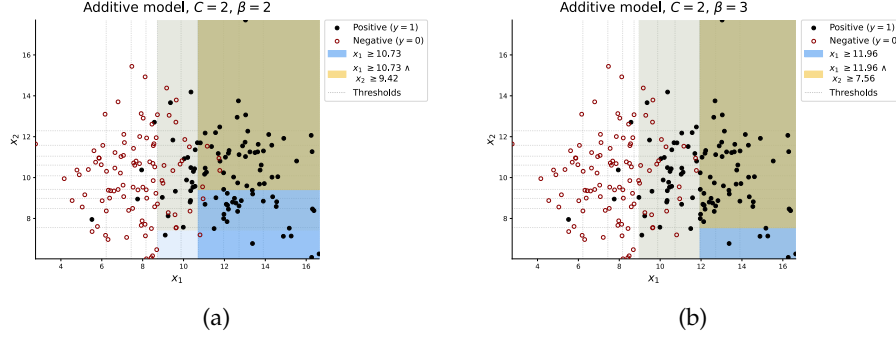


Figure 4.7: Solutions to an instance of Example 4.3.1 computed with the ramp loss model, configured with $C = 2, \beta = 2$ (a) and $C = 2, \beta = 3$ (b), respectively.

straint (4.1b) implies that the loss of a positive point decreases as the number of margins it is in increases. In particular, being in two or more margins can be equivalent to being far inside the covered region, which gives a loss of 0. This encourages the use of more than one rule. From $\beta = 2$ onwards, the ruleset consists of two rules, one of which is contained in the other. This is depicted in Figure 4.7. The objective is again much lower than the simple rule $x_1 \geq 9.88$. This phenomenon can again be attributed to the additive nature of the model.

Example 4.3.2 (Three clouds). Consider Example 4.3.1 with an additional positive cloud of 10 points, drawn from $\text{Normal}((5, 15), 1)$. An instance is depicted in Figure 4.8. We again solve the ramp loss model for parameters

$$C \in \{1, 2, 3\}, \quad \beta \in \{0, 0.5, 1, 2, 3, 4, 5\}, \quad D \in \{2\}.$$

For $C = 1$, the rule $x_1 \geq 9.71$ is found for all β up until and including $\beta = 4$. For $C = 1, \beta = 5$, the rule is $x_1 \geq 8.77$ instead. Notably, the margin covers

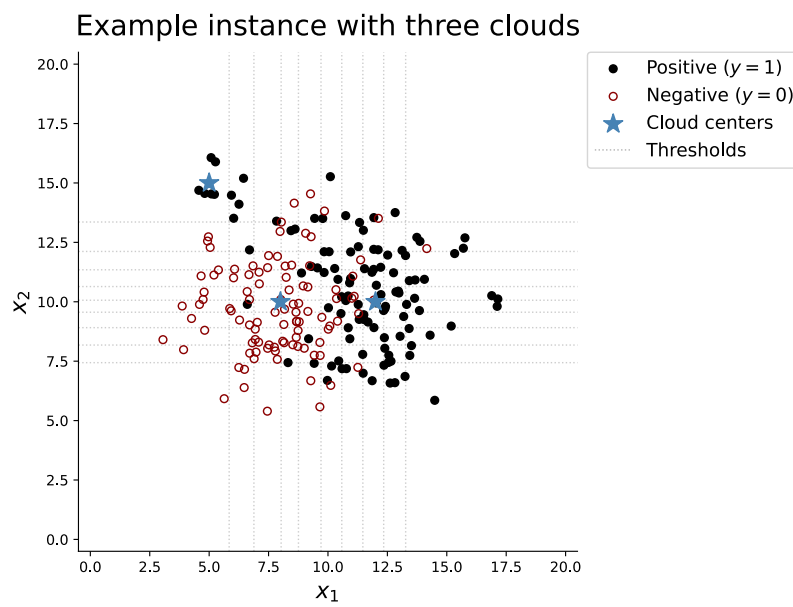


Figure 4.8: Instance of Example 4.3.2. The example consists of a negative cloud of 100 points, a positive cloud of 100 points and a positive cloud of 10 points. The point clouds have distributions $\text{Normal}((8, 10), 2)$, $\text{Normal}((12, 10), 2)$ and $\text{Normal}((5, 15), 1)$, respectively.

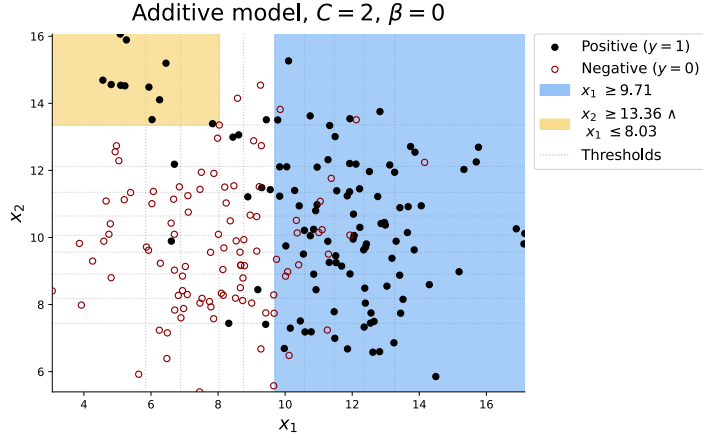


Figure 4.9: Solution to an instance of Example 4.3.2 computed with the ramp loss model, configured with $C = 2$, $\beta = 0$, $D = 2$.

every point in the positive cloud of 10 points in this case. For $C = 2$ and $\beta \in \{0, 0.5, 1\}$, the ruleset

$$x_1 \geq 9.71 \quad \vee \quad (x_1 \leq 8.03 \wedge x_2 \geq 13.36)$$

is found. This solution is depicted in Figure 4.9. For $C = 2$, $\beta = 2$ the resulting ruleset is different and may be considered reasonable. This ruleset is depicted in Figure 4.10a. For $C = 2$, $\beta > 2$, rulesets consist of overlapping rules and ignore the positive cloud of 10 points. An example is depicted in Figure 4.10b. For $C = 3$, the solutions are similar to the $C = 2$ case of Example 4.3.1, but with an extra rule to accommodate the additional positive point cloud. An example is shown in Figure 4.11.

In these two examples, the model behaves as desired when $\beta = 0$. When a margin is introduced, freedom to choose rules results in undesirable rulesets. The clearest example of this is Figure 4.10b, where a positive point cloud is ignored, because overlapping rules are beneficial in terms of loss, due to additivity in the model. From a practitioner's perspective, the problem is sometimes solved by limiting C and β . After all, for adequate choice of C and β , the model produces results considered reasonable. However, the results suggest that the model in its current form is not suitable for the problem at hand. Therefore, we introduce a version of the model where positive points no longer benefit from being in more than one margin. We refer to this model as the non-additive ramp loss model.

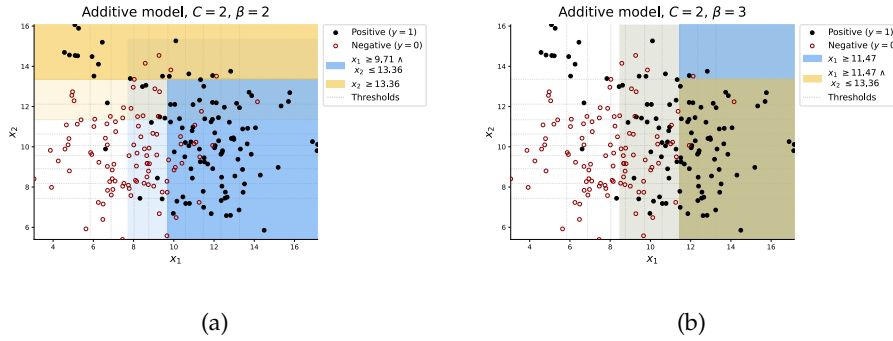


Figure 4.10: Solutions to an instance of Example 4.3.2 computed with the ramp loss model, configured with $C = 2, \beta = 2$ (a) and $C = 2, \beta = 3$ (b), respectively.

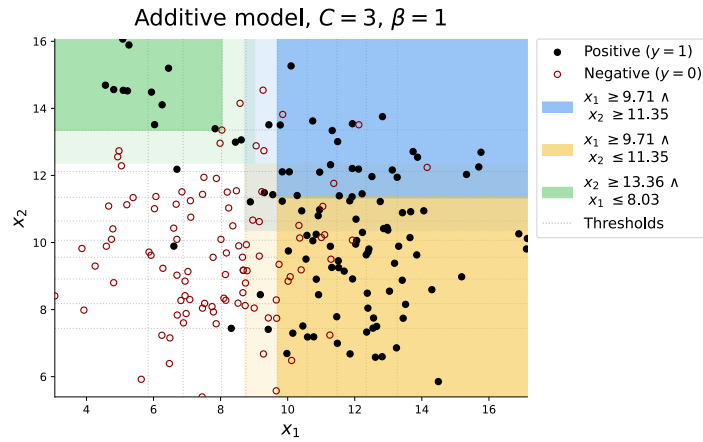


Figure 4.11: Solution to an instance of Example 4.3.2 computed with the ramp loss model, configured with $C = 3, \beta = 1, D = 2$.

4.4 Non-additive variant

In the non-additive ramp loss model, we wish to replace

$$\zeta_i + \sum_{k \in \mathcal{K}} \gamma_{ik} w_k \geq 2, \quad i \in \mathcal{P},$$

by

$$\zeta_i + \max_{k \in \mathcal{K}} \gamma_{ik} w_k \geq 2, \quad i \in \mathcal{P}.$$

which is equivalent to

$$\begin{aligned} \zeta_i - 2 &\geq -\max_{k \in \mathcal{K}} \gamma_{ik} w_k \\ &= \min_{k \in \mathcal{K}} -\gamma_{ik} w_k, \quad i \in \mathcal{P}. \end{aligned}$$

Define

$$\eta_{ik} = -\gamma_{ik}, \quad i \in \mathcal{P}, k \in \mathcal{K}.$$

Then one formulation of the non-additive ramp-loss model is

$$\min \quad \sum_{i \in \mathcal{P}} \zeta_i + \sum_{i \in \mathcal{N}} \sum_{k \in \mathcal{K}} \gamma_{ik} w_k \quad (4.15a)$$

$$\text{s.t.} \quad \zeta_i - 2 \geq \eta_{ik'} - \sum_{\substack{k \in \mathcal{K}, \\ \eta_{ik} < \eta_{ik'}}} (\eta_{ik'} - \eta_{ik}) w_k, \quad \forall i \in \mathcal{P}, k' \in \mathcal{K}, \quad (4.15b)$$

$$\sum_{k \in \mathcal{K}} c_k w_k \leq C, \quad (4.15c)$$

$$w \in \{0, 1\}^{\mathcal{K}}, \zeta \in [0, 2]^{\mathcal{P}}. \quad (4.15d)$$

The reduced cost of a rule is derived in Section B.3. However, this reduced cost cannot be easily modeled inside a MIP. Alternatively, we may consider the following formulation:

$$\min \quad \sum_{i \in \mathcal{P}} \zeta_i + \sum_{i \in \mathcal{N}} \sum_{k \in \mathcal{K}} \gamma_{ik} w_k \quad (4.16a)$$

$$\text{s.t.} \quad \zeta_i - 2 \geq \sum_{k \in \mathcal{K}} u_{ik} \eta_{ik}, \quad \forall i \in \mathcal{P}, \quad (4.16b)$$

$$\sum_{k \in \mathcal{K}} u_{ik} = 1, \quad \forall i \in \mathcal{P}, \quad (4.16c)$$

$$u_{ik} \leq w_k, \quad \forall i \in \mathcal{P}, k \in \mathcal{K}, \quad (4.16d)$$

$$\sum_{k \in \mathcal{K}} c_k w_k \leq C, \quad (4.16e)$$

$$w \in \{0, 1\}^{\mathcal{K}}, \zeta \in [0, 2]^{\mathcal{P}}, u \in [0, 1]^{\mathcal{P} \times \mathcal{K}}. \quad (4.16f)$$

While this formulation has a large number of variables, the u -variables need not be binary. The reduced cost of a variable w_k , $k \in \mathcal{K}$, is derived in Section B.3. Implementing column-and-row generation for this master problem

formulation is not trivial, as a rule corresponds to a w -variable and many u -variables.

Evidently, solving the relaxed non-additive ramp loss model exactly is difficult, as even the LP cannot be easily solved via column (and row) generation. A pragmatic approach is to rely solely on a pricing heuristic. Adapting the heuristic pricer to the non-additive model is possible, but requires substantial work. Therefore, we first explore whether the non-additive model gives better results on example instances. We consider again the setting and examples in Section 4.3. We use formulation (4.15), as it is easiest to implement within the existing implementation of the ramp loss model.

Example 4.4.1 (Two clouds, non-additive model). Consider again Example 4.3.1 and the tested parameters.

For $C = 1$ and any β , the results are identical to those of the additive model. For $C = 2$, $\beta = 0$, the ruleset differs and is depicted in Figure 4.12a. It consists of two rules, and the covered region is identical to the one-rule solution found for the additive model. The objective is also identical. To the model, both solutions are equally good, For classification tasks, they are also identical. If the smaller ruleset is preferred, for instance because of interpretability, then C could be chosen as low as possible so long as the performance, whichever way it is measured, does not decrease. Another solution could be to penalize ruleset complexity in the model. It is important to note that this solution, when considered in the additive model with $\beta = 0$, has the exact same objective value as Figure 4.5. Hence, the choice of this solution in the non-additive model is a coincidence rather than an artifact. In both the additive and non-additive model, the value of C is best kept small to prevent this.

For $C = 2$, $\beta = 0.5$ the solution is similar to Figure 4.6a and is shown in Figure 4.12b. For $C = 2$, $\beta = 1$, the solution contains $x_1 \geq 9.88$, but also another rule, as shown in Figure 4.12c. Removing this other rule from the ruleset, leaving just the rule $x_1 \geq 9.88$, results in the same objective. Hence, one may want to prune solutions. One way of pruning solutions is to consider all subsets of the ruleset, and pick the smallest subset that has the same objective value as the full ruleset. Alternatively, one could again limit C or penalize rule complexity.

Notably, for $\beta > 0.5$, the non-additive model seems less inclined to produce undesirable rulesets than the additive model. While the additive model yields two-rule solutions for $C = 2$, $\beta > 0.5$, the non-additive yields a one-rule solution either immediately or after pruning. This can be partly attributed to the non-additivity that was introduced. Another factor is the negative points' loss, which is calculated in the same way in either model. The loss for negative points is additive, hence using more rules is bad for negative points in or around the covered region.

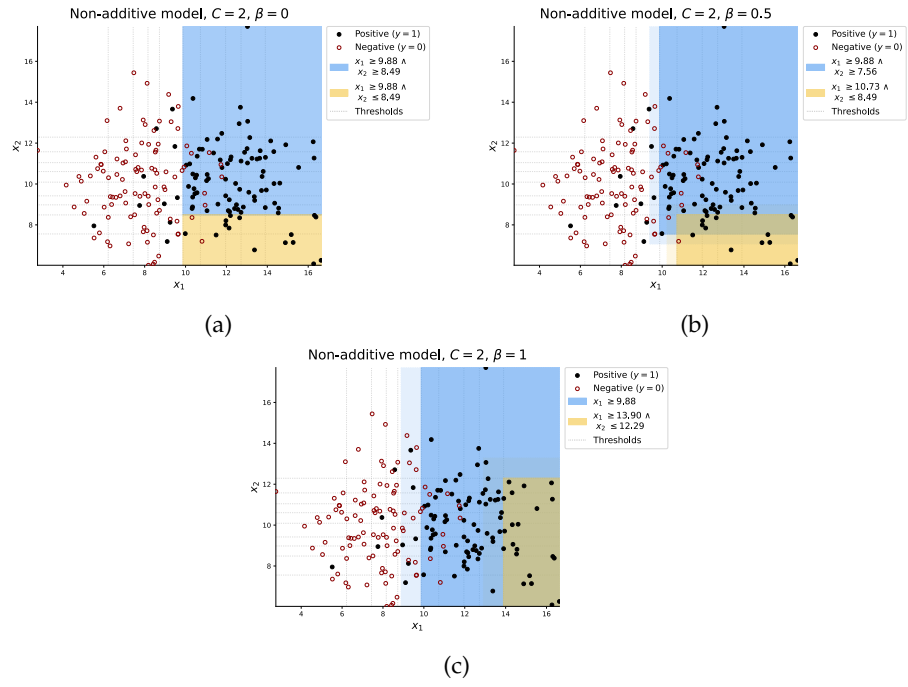


Figure 4.12: Solutions to an instance of Example 4.3.1 computed with the ramp loss model, configured with $C = 2, \beta = 0.5$ (a) and $C = 2, \beta = 1$ (b), respectively.

We conclude that the non-additive ramp loss model behaves better than the additive ramp loss model. In practice, we perform pricing on the additive variant. Then, we use the non-additive variant inside the branching stage. This effectively means we take the columns generated using the additive variant, add these columns to an IP of the non-additive variant, and solve.

4.5 Implementation

The custom solver described in Section 2.3 is capable of solving the ramp loss model. This means there is another option, namely the model type, which is either “Boolean” or “Threshold”. The model type “Threshold” solves both the threshold model (Chapter 3) and the ramp loss model, as the threshold model is a special case of the ramp loss model, precisely the case $\beta = 0$.

The implementation for both model types share relevant parts of the code-base. Notably, both model types use the same basis for the heuristic pricer. The key difference is how the reduced cost is computed inside the search tree. We will now discuss how this is done for the ramp loss model. Let $k_{\text{parent}}, k_{\text{child}}$ be the rules corresponding to some parent and child node, respectively. Then the reduced cost of the child node is given by

$$\hat{\rho}_{k_{\text{child}}} = \sum_{i \in \mathcal{N}} \gamma_{ik_{\text{child}}} - \sum_{i \in \mathcal{P}} \mu_i \gamma_{ik_{\text{child}}} + \lambda c_{k_{\text{child}}}. \quad (4.17)$$

We now wish to express $\gamma_{ik_{\text{child}}}$ in terms of $\gamma_{ik_{\text{parent}}}$. Ideally, we can then store γ -values cleverly in the search tree, and compute $\hat{\rho}_{k_{\text{child}}}$ from $\hat{\rho}_{k_{\text{parent}}}$ with relatively little effort. Denote by l the clause that is added in k_{child} with respect to k_{parent} . Recall the definition of γ_{ik} in (4.9). Evidently,

$$\gamma_{ik_{\text{child}}} \leq \gamma_{ik_{\text{parent}}}.$$

Precisely, observe that

$$\gamma_{ik_{\text{child}}} = \gamma_{ik_{\text{parent}}} - (\gamma_{ik_{\text{parent}}} - \gamma_{il}) \mathbb{I}\{\gamma_{il} < \gamma_{ik_{\text{parent}}}\}. \quad (4.18)$$

For convenience, we denote

$$\Delta \gamma_{ik} = (\gamma_{ik_{\text{parent}}} - \gamma_{il}) \mathbb{I}\{\gamma_{il} < \gamma_{ik_{\text{parent}}}\} \quad (4.19)$$

It follows that

$$\hat{\rho}_{k_{\text{child}}} = \hat{\rho}_{k_{\text{parent}}} - \sum_{i \in \mathcal{N}} \Delta \gamma_{ik_{\text{child}}} + \sum_{i \in \mathcal{P}} \Delta \mu_i \gamma_{ik_{\text{child}}} + \lambda. \quad (4.20)$$

There are two important cases in which $\Delta \gamma_{ik} = 0$. First, if $\gamma_{ik_{\text{parent}}} = 0$, as γ_{il} cannot be lower. Second, if $\gamma_{il} = 2$, as $\gamma_{ik_{\text{parent}}}$ cannot be higher. Define

$$\bar{\mathcal{I}}_k := \{i \in \mathcal{I} : \gamma_{ik} > 0\} \quad k \in \mathcal{K}. \quad (4.21)$$

In other words, $\overline{\mathcal{I}}_k$ is the set of points that are covered by rule k or are in the margin. Define $\overline{\mathcal{N}}$ and $\overline{\mathcal{P}}$ analogously. Then, instead of summing over $\mathcal{N}$ and $\mathcal{P}$, we may sum over

$$\overline{\mathcal{N}} \cap \{i \in \mathcal{N} : \gamma_{il} < 2\} \quad \text{and} \quad \overline{\mathcal{P}} \cap \{i \in \mathcal{P} : \gamma_{il} < 2\},$$

respectively. Observe that

$$\overline{\mathcal{I}}_{k_{\text{child}}} \subseteq \overline{\mathcal{I}}_{k_{\text{parent}}}.$$

and

$$\overline{\mathcal{I}}_{k_{\text{child}}} = \overline{\mathcal{I}}_{k_{\text{parent}}} \setminus \{i \in \mathcal{I} : \gamma_{il} = 0\}.$$

We store $\overline{\mathcal{I}}_k$ for each node k in the search tree. Furthermore, we do not store every γ_{ik} , as this would require many variables in each node of the search tree. Instead, we store the key-value pair $(i, \gamma_{ik_{\text{child}}})$ at node k_{child} if $\gamma_{ik_{\text{child}}}$ changed with respect to the parent. Hence, finding the value of $\gamma_{ik_{\text{parent}}}$ requires traversing up the search tree until there is a hit.

4.6 Methodology

We conduct the experiments described in Section 3.3 again, but now the non-additive variant for various margin widths $\beta > 0$. In particular, we consider

$$\beta = 0, 0.2, 0.4, 0.6, 0.8, 1$$

We choose the range 0 to 1 because the standard deviation of most TF is around 2. We compare these results with the earlier results, which will be indicated by $\beta = 0$.

We do not expect train accuracy to improve. After all, the model directly maximizing accuracy is the original master problem (2.4). Now we are considering a new variant, based on the Hamming loss, in an attempt to obtain qualitatively better results, but not necessarily better train accuracy.

Instead, we will look at the qualitative results, in particular the ruleset frequency under cross-validation. We also look at generalization (train and test accuracy). We aim for a better generalization, i.e., a higher test accuracy.

4.7 Results

The train and test accuracies are given in Figure D.1. From this plot, there is no clear improvement in test accuracy for $\beta > 0$. Increasing β does appear to dampen the effect of C on train accuracy. This may be interpreted as the model being less inclined to overfit for large C . It may also indicate that the model is less capable of learning altogether.

Table 4.1

Q_y	C	D	TF combination	Frequency
0.5	1	2	RELA $\wedge$ STAT2	10
		3	FOS $\wedge$ RELA $\wedge$ STAT2	10
0.6	1	2	RELA $\wedge$ STAT2	5
			FOSL1 $\wedge$ RELA	5
		3	FOS $\wedge$ RELA $\wedge$ STAT2	4
			FOSL1 $\wedge$ RELA $\wedge$ STAT2	3
			FOSL1 $\wedge$ IRF1 $\wedge$ RELA	2
			FOS $\wedge$ FOSL1 $\wedge$ RELA	1

Based on the plot, we study $\beta = 0.6$ in more detail. The ruleset frequency table is given in Table 4.1. For $Q_y = 0.5$, the ruleset found is now consistent among all splits. The opposite is true for $Q_y = 0.6$. Interestingly, the reverse was true for $\beta = 0$, as seen in Table 3.3.

It appears therefore that the introduction of the ramp loss, which lead to non-additive model, did not have the desired effect.

Chapter 5

Additional experiments

5.1 Tissue status

It is worth investigating whether there is a difference in results when considering only tumors or only metastases. A metastasis tends to have more genetic mutations than a tumor. If more mutations cause more complicated and more unique behavior within the cell, it may be harder to deduce general rules governing PD-L1 expression in metastases. If true, focusing on one or the other independently could be useful.

A related hypothesis is that metastases have higher PD-L1 expression due to their tendency for more mutations. To investigate this, the distribution of PD-L1 expression per tissue type is shown in Figure 5.1. Note that cell lines belonging to Papiloma or Normal are disregarded due to a lack of samples. The density graphs have a similar shape. The mean PD-L1 expression per tissue type is also similar. Alternatively, if the hypothesis is true, we expect to see relatively many metastases among samples with high PD-L1 expression. The tissue status distribution per PD-L1 expression quantile bin is depicted in Figure 5.2. This histogram does not suggest the balance of tumors and metastases changes significantly with PD-L1 expression. Thus, the results suggest that the hypothesis is false.

Still, there may be a difference in the behavior of the model between tumors and metastases. To test this, the greedy method was used on Broad for tumors and metastases separately. Both the tumors in Broad and the metastases in Broad were treated as separate datasets, each undergoing independent discretization, and were subjected to the algorithm independently. We consider $Q_y = 0.5, 0.6$ and $D = 2$.

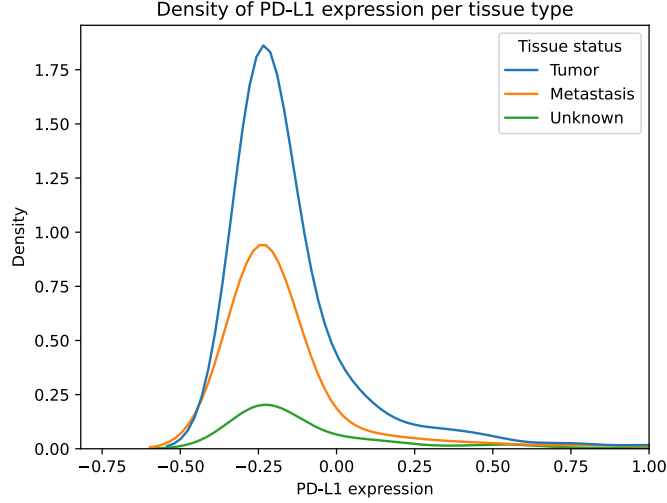


Figure 5.1: Density of PD-L1 expression for tissue types Tumor, Metastasis and Unknown in the Broad dataset.

The results are shown in Table 5.1 through Table 5.4. In all four cases, there is effectively no value in adding more than one rule. Recall that after the first rule is found, all covered points are stripped away and the best rule for the remaining points is found through enumeration. As such, there simply no effective rule for the leftover points. Accuracy is slightly higher for the tumor instances. The first rules found in each of the four variants are all qualitatively different. Therefore, the only conclusion that may be drawn from this experiment is that PD-L1 expression is slightly more easily captured in some ruleset for tumors. However, this says nothing about the utility of the rulesets found.

Table 5.1: Results of the greedy method for tumors, $Q_y = 0.5$.

τ	# in $\mathcal{I}_\tau$	# in $\mathcal{P}_\tau$	# covered by k_τ	# in $\mathcal{P}_\tau$ covered by k_τ	Acc.	Rule
1	507	254	289	203	0.73	$\{\text{RELA} \geq -1.422 \text{ Q}(0.201) \wedge \text{STAT2} \geq -0.424 \text{ Q}(0.400)\}$
2	218	51	12	9	0.74	$\{\text{AR} \geq -1.455 \text{ Q}(0.697) \wedge \text{FOSL1} \geq 0.520 \text{ Q}(0.899)\}$
3	206	42	12	9	0.75	$\{\text{STAT2} \geq -1.265 \text{ Q}(0.201) \wedge \text{FOXO1} \leq -1.516 \text{ Q}(0.102)\}$

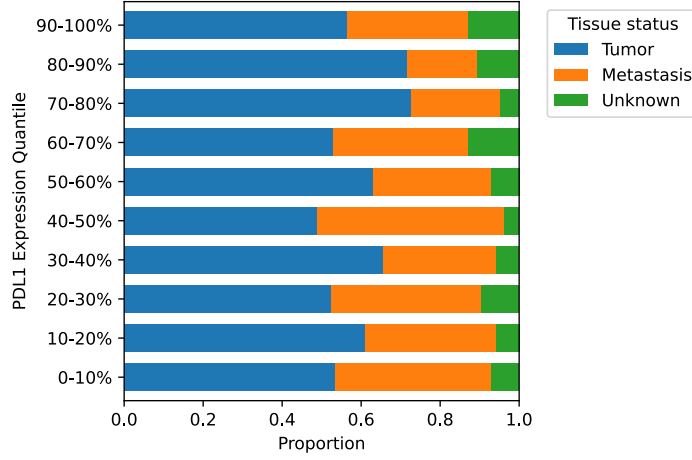


Figure 5.2: Distribution of tissue type within various quantile bins of PD-L1 expression in the Broad dataset

Table 5.2: Results of the greedy method for tumors, $Q_y = 0.6$.

τ	# in $\mathcal{I}_\tau$	# in $\mathcal{P}_\tau$	# covered by k_τ	# in $\mathcal{P}_\tau$ covered by k_τ	Acc.	Rule
1	507	203	171	124	0.75	$\{\text{FOSL1} \geq 0.156 \text{ Q}(0.600) \wedge \text{STAT2} \geq -0.698 \text{ Q}(0.300)\}$
2	336	79	16	14	0.78	$\{\text{IRF1} \geq 0.432 \text{ Q}(0.699) \wedge \text{SPI1} \leq 0.005 \text{ Q}(0.500)\}$
3	320	65	12	9	0.79	$\{\text{IRF1} \geq 1.892 \text{ Q}(0.900) \wedge \text{STAT2} \geq 1.024 \text{ Q}(0.900)\}$

5.2 Gene expression as input

The use of the DecoupleR model ([6, 5]) raises some suspicion. The model itself assumes that TF act individually to regulate a set of target genes. A prior knowledge matrix, in our case based on CollecTRI, describes which TF are known to up- or downregulate which genes. The gene expressions of all the TF’s target genes are then use to “reverse engineer” (estimate) the TF activity. DecoupleR’s output is taken as an undisputed truth onwards. It is unclear whether combinatoric effects can be observed from output of a model that does not assume combinatoric behavior.

Furthermore, the rules we have found may fail to capture real-life phenomena and instead reflect information from the prior knowledge matrix. Due

Table 5.3: Results of the greedy method for metastases, $Q_y = 0.5$.

τ	# in $\mathcal{I}_\tau$	# in $\mathcal{P}_\tau$	# covered by k_τ	# in $\mathcal{P}_\tau$ covered by k_τ	Acc.	Rule
1	272	136	133	90	0.67	$\{\text{IRF1} \geq -1.444 \text{ Q}(0.202) \wedge$ $\text{RELA} \geq -0.436 \text{ Q}(0.500)\}$
2	139	46	21	16	0.71	$\{\text{SPI1} \leq -2.184 \text{ Q}(0.201) \wedge$ $\text{GATA2} \leq -0.702 \text{ Q}(0.302)\}$
3	118	30	7	6	0.73	$\{\text{FOSL1} \geq 0.203 \text{ Q}(0.797) \wedge$ $\text{NFATC1} \geq -0.556 \text{ Q}(0.500)\}$

Table 5.4: Results of the greedy method for metastases, $Q_y = 0.6$.

τ	# in $\mathcal{I}_\tau$	# in $\mathcal{P}_\tau$	# covered by k_τ	# in $\mathcal{P}_\tau$ covered by k_τ	Acc.	Rule
1	272	109	75	54	0.72	$\{\text{FOSL1} \geq -0.057 \text{ Q}(0.500) \wedge$ $\text{RELA} \geq 0.009 \text{ Q}(0.599)\}$
2	197	55	12	10	0.75	$\{\text{GATA2} \geq -0.362 \text{ Q}(0.401) \wedge$ $\text{FOSL1} \leq -0.755 \text{ Q}(0.102)\}$
3	185	45	6	6	0.77	$\{\text{NFKB1} \geq -2.719 \text{ Q}(0.103) \wedge$ $\text{SPI1} \leq -2.431 \text{ Q}(0.103)\}$

to the construction of the DecoupleR model, TFs that target similar genes will have similar activity estimates. This may partly explain the high cross-correlations observed in Figure A.1a. As a result, even if TF activity differs in reality, the DecoupleR model can make TF activities hard to distinguish. This, in turn, can make finding meaningful rules difficult. Instead, the rules may represent the prior knowledge matrix, meaning of interest nothing is learned.

As a curiosity, we use the threshold model (Chapter 3) on the raw gene expression data instead. While this is not expected to yield a result with a realistic interpretation, it can possibly give insight how well the model behaves.

Figure D.2 shows the results, in the same way as Figure 3.1 does. The interpretable results for $C = 1$ and $D = 1, 2, 3$ are given in Table D.3. The interpretable results for $C = 2$ are given in Table D.4.

Make sure these have the $D = 1$ results.

It follows that the accuracy achieved is higher compared to the experiments with TF activity estimates. Furthermore, qualitatively different results are observed. This indicates that gene expression is indeed not equivalent to TF activity.

5.3 Transcription Factor Exclusion

Evidently, learning logical rules from bulk transcriptomics is not easy. Rather than focussing on the rulesets produced and their consistency, we will now attempt to identify TF that are “important”. That is, TF that are essential to classifying PD-L1 expression “well” with rulesets.

To do this, we argue by exclusion: if a TF being excluded from the dataset causes performance to drop, then this TF may be considered important.

Consider $C = 1, D = 2$. If the best possible ruleset is, for instance,

$$\text{TF A} \wedge \text{TF B},$$

then removing any TF other to A and B from the dataset will not remove this solution. Therefore, removing any TF unequal to A or B will have no effect on the maximum performance. In other words, the only removals that have any effect on performance are either removing A or removing B.

We use this idea to build a tree. First, the best ruleset is found. For each TF in the ruleset, a branch is created in which it is excluded. Then the best ruleset is found and the process repeats. We consider

$$C = 1, \quad D = 1, 2, \quad \text{and} \quad Q_y = 0.5, 0.6.$$

Since it is feasible to solve these cases to optimality with the greedy method, this method is used.

For the results, see Table D.5. The reduction in accuracy upon exclusion of a TF is small. Still, these tables may provide some insight to a biologist.

Chapter 6

Performance on holdout dataset

In this chapter, we use some of the rules found for the Broad dataset to predict PD-L1 expression in the Sanger dataset. The performance of these rulesets is also compared to the corresponding optimal ruleset for the Sanger dataset itself.

There are two ways of evaluating a ruleset produced from a train dataset on a test dataset:

- **Absolute inference:** The ruleset is characterized by the absolute values of the thresholds. For instance, if the clause is

$$\text{STAT2} \geq -0.20 \text{ } Q(0.5),$$

then

$$\text{STAT2} \geq -0.20$$

is used for the test dataset.

- **Relative inference:** The ruleset is characterized by the quantile level with respect to the observed TF activity values in the test. For instance if the clause is

$$\text{STAT2} \geq -0.20 \text{ } Q(0.5),$$

then

$$\text{STAT2} \geq a$$

is used for the test dataset, where a is the 0.5 quantile of STAT2 activity in the test dataset.

Table 6.1: Performance of various one-rule rulesets on both Broad and Sanger. Performance for absolute inference is indicated with “(Abs.)” and performance for relative inference is indicated with “(Rel.)”

			Acc. Broad	Acc. Sanger (Abs.)	Acc. Sanger (Rel.)	Ruleset
Q_y	C	D				
0.5	1	1	0.69	0.68	0.71	$\{\text{STAT2} \geq -0.20 Q(0.50)\}$
		2	0.71	0.67	0.71	$\{\text{STAT2} \geq -0.20 Q(0.50) \wedge \text{FOS} \geq -1.26 Q(0.20)\}$
		3	0.71	0.64	0.71	$\{\text{STAT2} \geq -0.20 Q(0.50) \wedge \text{FOS} \geq -1.26 Q(0.20) \wedge \text{IRF1} \geq -1.59 Q(0.10)\}$
0.6	1	1	0.70	0.57	0.63	$\{\text{FOSL1} \geq 0.41 Q(0.70)\}$
		2	0.73	0.62	0.70	$\{\text{FOSL1} \geq 0.13 Q(0.60) \wedge \text{STAT2} \geq -0.74 Q(0.30)\}$
		3	0.74	0.61	0.71	$\{\text{FOSL1} \geq 0.13 Q(0.60) \wedge \text{STAT2} \geq -0.74 Q(0.30) \wedge \text{FOS} \geq -0.78 Q(0.30)\}$

During cross-validation experiments, absolute inference was used, as both train and test datasets were subsets of the same dataset. It may be that Sanger is biased with respect to Broad, for instance due to a difference in measurement techniques or lab environment. Therefore, relative inference may be the better choice.

The idea is not to evaluate every result obtained on the Sanger dataset, but only those deemed most promising.

What can be said about these results that is meaningful?

Chapter 7

Recommendations

Based on the findings, we present three suggestions for future research, ranging from very related to our work to a completely novel approach.

In Chapter 4 we apply a ramp loss based on the distance of a point to a ruleset. An alternative is to define an *importance score*

$$\chi_i = \left[\frac{1}{\beta} |(\text{PD-L1 expression})_i - \alpha| \right]_0^1,$$

where $[\cdot]$ is the clipping operator, α is the gene expression threshold and β is the margin width. Then, we consider

$$\sum_{i \in \mathcal{P}} \chi_i \zeta_i + \sum_{i \in \mathcal{N}} \chi_i \zeta_i$$

as the loss function in the master IP (2.6). We may also consider a Hamming loss equivalent. This effectively means that samples whose gene expression is closer to the target threshold are weighed less heavily. This is arguably a much simpler adjustment of the boolean model or threshold model that is similar conceptually. In fact, it may avoid many of the artifacts observed in the ramp loss model and difficulties encountered in its implementation.

Another suggestion is to re-evaluate use of DecoupleR as unquestioned input data. DecoupleR itself is an estimation method. In particular, one that assumes that TF affect target genes individually. Rather than treating TF activity estimation as data pre-processing, a suggestion is to unite TF activity score estimation and the search for TF combinations into a single, novel method. Perhaps a united approach, one that assumes combinatorial TF regulation from raw data onwards, can yield better results.

Finally, it is suggested to use statistical methods to search for statistically significant concurrent TF activity, rather than attempt to predict PD-L1 expression using discrete, deterministic classifiers. An example of such a method, albeit for a different use case, is LAMP [24].

Chapter 8

Conclusion

TODO

Use of Artificial Intelligence

The author believes that Artificial Intelligence (AI) has been a useful tool for learning new skills, debugging and reducing work load throughout this project. At the start of this project, the author did not know how to program in C++. AI systems, in particular Copilot and Claude, were used to learn about the language's syntax and features. Moreover, the systems were helpful in learning how to use the SCIP framework. The codebase was then designed and written by hand. These AI systems were also used to identify errors and bugs, as well as write some utility functions. On the data analysis side, the systems proved capable of writing and tweaking code for making plots and tables, ensuring that these plots look as intended. Overall, the creation of visuals was strongly aided by these systems. The AI systems also contributed, to an extent, to finding relevant literature for Section 4.2. This report was written entirely by hand. These AI systems only played a role in finding typing errors and inconsistencies, which were then resolved by hand.

Bibliography

- [1] David Aha. *Tic-Tac-Toe Endgame*. Published: UCI Machine Learning Repository. 1991. DOI: <https://doi.org/10.24432/C5688J>.
- [2] Elaine Angelino et al. *Learning Certifiably Optimal Rule Lists for Categorical Data*. Issue: arXiv:1704.01701. Aug. 3, 2018. DOI: 10.48550/arXiv.1704.01701. arXiv: 1704.01701 [stat]. URL: <http://arxiv.org/abs/1704.01701> (visited on 03/03/2026).
- [3] Alper Atamturk and Vishnu Narayanan. *Submodular Function Minimization and Polarity*. Issue: arXiv:1912.13238. Dec. 8, 2020. DOI: 10.48550/arXiv.1912.13238. arXiv: 1912.13238 [math.OC]. URL: <http://arxiv.org/abs/1912.13238> (visited on 05/26/2026).
- [4] Alper Atamtürk, George L. Nemhauser, and Martin W.P. Savelsbergh. “The mixed vertex packing problem”. In: *Mathematical Programming* 89.1 (Nov. 1, 2000), pp. 35–53. ISSN: 1436-4646. DOI: 10.1007/s101070000154. (Visited on 08/25/2026).
- [5] Pau Badia-i-Mompel. *Transcription factor activity inference in bulk RNA-seq*. decoupleR. URL: https://saezlab.github.io/decoupleR/articles/tf_bk.html (visited on 07/29/2026).
- [6] Pau Badia-i-Mompel et al. “decoupleR: ensemble of computational methods to infer biological activities from omics data”. In: *Bioinformatics Advances* 2.1 (Mar. 8, 2022), vbac016. ISSN: 2635-0041. DOI: 10.1093/bioadv/vbac016. (Visited on 07/29/2026).
- [7] Pietro Belotti. “Spatial branching for a special class of convex MIQO problems”. In: *Optimization Letters* 18.8 (Nov. 2024), pp. 1757–1770. ISSN: 1862-4472, 1862-4480. DOI: 10.1007/s11590-024-02143-8. (Visited on 04/22/2026).
- [8] J. Paul Brooks. “Support Vector Machines with the Ramp Loss and the Hard Margin Loss”. In: *Operations Research* 59.2 (2011), pp. 467–479. ISSN: 0030-364X. (Visited on 04/22/2026).
- [9] *Cell Model Passports*. URL: <https://cellmodelpassports.sanger.ac.uk>.
- [10] Corinna Cortes and Vladimir Vapnik. “Support-vector networks”. In: *Machine Learning* 20.3 (Sept. 1, 1995), pp. 273–297. ISSN: 1573-0565. DOI: 10.1007/BF00994018. (Visited on 04/28/2026).

- [11] Sanjeeb Dash, Oktay Günlük, and Dennis Wei. *Boolean Decision Rules via Column Generation*. Issue: arXiv:1805.09901. Aug. 5, 2020. DOI: 10.48550/arXiv.1805.09901. arXiv: 1805.09901[cs]. URL: <http://arxiv.org/abs/1805.09901> (visited on 03/03/2026).
- [12] Christopher Hojny et al. *The SCIP Optimization Suite 10.0*. Nov. 24, 2025. URL: <https://optimization-online.org/?p=32699> (visited on 08/09/2026).
- [13] Janosi Janosi Andras et al. *Heart Disease*. Published: UCI Machine Learning Repository. 1989. DOI: <https://doi.org/10.24432/C52P4X>.
- [14] Kelly, Markelle, Longjohn, Rachel, and Nottingham, Kolby. *The UCI Machine Learning Repository*. URL: <http://www.uci.edu/~mlr/> (visited on 03/03/2026).
- [15] Simge Küçükyavuz and Qimeng Yu. *Mixed-Integer Programming Approaches to Generalized Submodular Optimization and its Applications*. Apr. 2, 2023. URL: <https://arxiv.org/abs/2304.00479v2> (visited on 05/24/2026).
- [16] Connor Lawless. *conlaw/FairCG*. original-date: 2022-08-01T12:29:46Z. Mar. 20, 2026. URL: <https://github.com/conlaw/FairCG> (visited on 08/09/2026).
- [17] Connor Lawless et al. *Interpretable and Fair Boolean Rule Sets via Column Generation*. Issue: arXiv:2111.08466. Sept. 18, 2023. DOI: 10.48550/arXiv.2111.08466. arXiv: 2111.08466[cs]. URL: <http://arxiv.org/abs/2111.08466> (visited on 03/03/2026).
- [18] J.G.M. van der Linden. "Optimal Decision Trees: Algorithms and Applications". PhD thesis. Delft University of Technology, 2026. DOI: 10.4233/UUID:6DF25AA3-838D-4208-A6CC-83F95D8417F9. URL: <https://resolver.tudelft.nl/uuid:6df25aa3-838d-4208-a6cc-83f95d8417f9> (visited on 08/28/2026).
- [19] Volker Lohweg. *Banknote Authentication*. Published: UCI Machine Learning Repository. 2012. DOI: <https://doi.org/10.24432/C55P57>.
- [20] L. Lovász. "Submodular functions and convexity". In: *Mathematical Programming The State of the Art: Bonn 1982*. Ed. by Achim Bachem, Bernhard Korte, and Martin Grötschel. Berlin, Heidelberg: Springer, 1983, pp. 235–257. ISBN: 978-3-642-68874-4. DOI: 10.1007/978-3-642-68874-4_10. URL: https://doi.org/10.1007/978-3-642-68874-4_10 (visited on 05/26/2026).
- [21] Dieudonne van der Meer et al. "Cell Model Passports-a hub for clinical, genetic and functional datasets of preclinical cancer models". In: *Nucleic Acids Research* 47 (D1 Jan. 8, 2019), pp. D923–D929. ISSN: 1362-4962. DOI: 10.1093/nar/gky872.
- [22] Bendi Ramana and N. Venkateswarlu. *ILPD (Indian Liver Patient Dataset)*. Published: UCI Machine Learning Repository. 2022. DOI: <https://doi.org/10.24432/C5D02C>.
- [23] V. Sigillito et al. *Ionosphere*. Published: UCI Machine Learning Repository. 1989. DOI: <https://doi.org/10.24432/C5W01B>.
- [24] Aika Terada et al. "Statistical significance of combinatorial regulations". In: *Proceedings of the National Academy of Sciences of the United States of America* 110.32 (Aug. 6, 2013), pp. 12996–13001. ISSN: 1091-6490. DOI: 10.1073/pnas.1302233110.

- [25] V. N. Vapnik. *Statistical learning theory*. Adaptive and learning systems for signal processing, communications, and control. New York: Wiley, 1998. xxiv, 736. ISBN: 978-0-471-03003-4 978-81-265-2892-9.
- [26] Wolberg, William et al. *Breast Cancer Wisconsin (Diagnostic)*. Published: UCI Machine Learning Repository. 1993. DOI: <https://doi.org/10.24432/C5DW2B>.
- [27] I-Cheng Yeh. *Blood Transfusion Service Center*. Published: UCI Machine Learning Repository. 2008. DOI: <https://doi.org/10.24432/C5GS39>.
- [28] Yaping Zhang and Junke Zheng. "Functions of Immune Checkpoint Molecules Beyond Immune Evasion". In: *Regulation of Cancer Immune Checkpoints: Molecular and Cellular Mechanisms and Therapy*. Ed. by Jie Xu. Singapore: Springer, 2020, pp. 201–226. ISBN: 978-981-15-3266-5. DOI: 10.1007/978-981-15-3266-5_9. URL: https://doi.org/10.1007/978-981-15-3266-5_9 (visited on 08/26/2026).

Appendix A

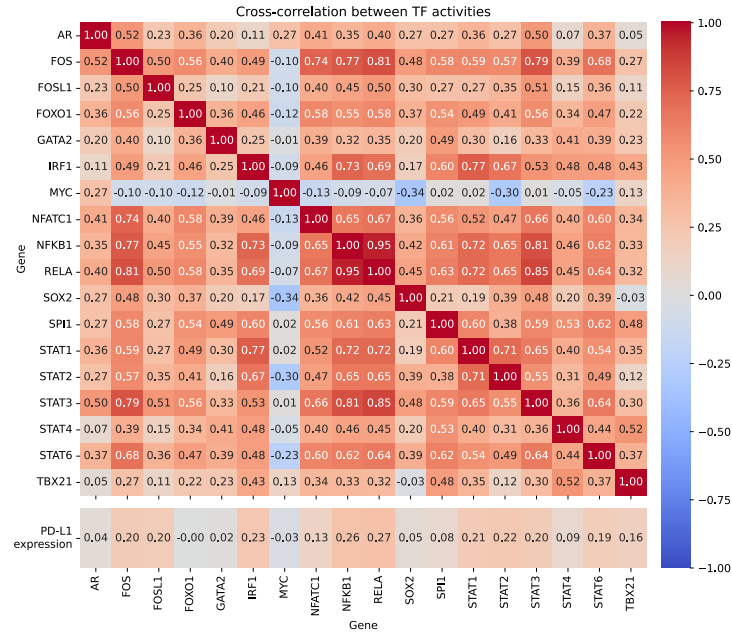
Data

Table A.1: Descriptive statistics for Broad dataset.

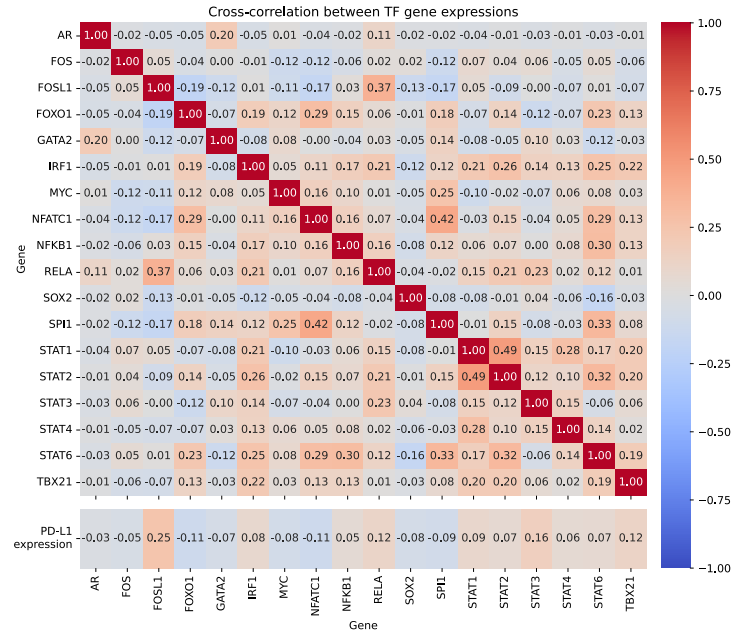
Activity Gene	Mean	Std.	Min.	25% quantile	50% quantile	75% quantile	Max.
AR	-1.55	1.72	-5.33	-2.75	-1.84	-0.67	8.04
FOS	0.29	1.87	-3.81	-0.98	0.01	1.33	9.63
FOSL1	0.18	1.02	-3.89	-0.41	-0.04	0.58	7.08
FOXO1	-0.10	1.09	-2.97	-0.87	-0.24	0.55	4.97
GATA2	0.57	1.46	-2.23	-0.31	0.31	1.11	9.69
IRF1	0.27	1.67	-3.98	-0.88	0.01	1.11	12.14
MYC	-4.73	3.32	-14.67	-7.09	-4.81	-2.54	5.55
NFATC1	0.39	1.41	-3.60	-0.53	0.13	1.03	10.30
NFKB1	0.65	2.28	-4.27	-0.87	0.35	1.97	11.18
RELA	0.35	2.36	-4.24	-1.33	0.03	1.72	10.51
SOX2	0.54	1.14	-3.68	-0.29	0.45	1.23	7.21
SPI1	0.72	2.36	-4.70	-0.86	0.36	1.74	14.45
STAT1	-0.94	1.94	-4.59	-2.24	-1.18	0.07	14.90
STAT2	0.11	1.36	-2.25	-0.85	-0.20	0.80	8.36
STAT3	0.14	1.82	-3.90	-1.15	-0.17	1.15	8.81
STAT4	0.58	0.98	-1.74	-0.01	0.45	0.99	9.28
STAT6	0.81	1.45	-2.60	-0.20	0.59	1.57	7.99
TBX21	0.23	1.16	-2.80	-0.37	-0.03	0.54	11.84
PDL1 expression	-0.12	0.33	-0.26	-0.26	-0.24	-0.14	3.55

Table A.2: Descriptive statistics for Sanger dataset.

Activity Gene	Mean	Std.	Min.	25% quantile	50% quantile	75% quantile	Max.
AR	2.21	2.16	-3.78	0.70	2.12	3.49	9.41
FOS	0.41	2.20	-4.39	-1.15	0.17	1.74	9.27
FOSL1	0.07	1.35	-3.32	-0.75	-0.17	0.60	7.50
FOXO1	0.30	1.31	-2.70	-0.59	0.09	1.10	5.76
GATA2	-0.40	1.75	-2.82	-1.29	-0.73	0.07	18.85
IRF1	-0.02	2.79	-4.25	-1.89	-0.51	1.22	14.12
MYC	5.21	3.54	-8.61	2.73	4.84	7.38	15.90
NFATC1	-0.05	1.42	-3.12	-0.98	-0.29	0.78	5.11
NFKB1	0.06	2.82	-5.54	-1.83	-0.13	1.57	10.05
RELA	0.54	2.95	-5.21	-1.71	0.15	2.41	10.56
SOX2	-0.33	1.75	-3.64	-1.41	-0.47	0.47	13.07
SPI1	-0.30	2.50	-5.15	-1.86	-0.69	0.80	11.63
STAT1	1.56	2.94	-3.97	-0.56	1.29	3.06	14.07
STAT2	0.22	2.00	-2.75	-1.20	-0.27	1.11	12.45
STAT3	0.54	2.17	-4.08	-0.96	0.27	1.93	8.17
STAT4	-0.56	1.26	-2.20	-1.32	-0.88	-0.23	8.06
STAT6	-0.53	1.53	-3.67	-1.54	-0.77	0.26	8.43
TBX21	-0.23	1.56	-2.26	-1.13	-0.63	0.05	10.71
PDL1 expression	0.31	1.70	-0.26	-0.25	-0.19	0.06	14.40



(a) TF activity estimates



(b) TF gene expression

Figure A.1: Cross-correlation matrices for TF activity estimates (a) and TF gene expression (b), including correlation with PD-L1 expression (bottom row of each matrix).

Appendix B

Reduced costs

B.1 Boolean model

To derive reduced cost (2.7), recall that the dual of the linear program (LP)

$$\begin{aligned} \min \quad & gx \\ \text{s.t.} \quad & Ax \geq b, \\ & x \geq 0, \end{aligned} \tag{B.1}$$

is given by

$$\begin{aligned} \max \quad & yb \\ \text{s.t.} \quad & yA \leq g, \\ & y \geq 0. \end{aligned} \tag{B.2}$$

Denote by A_k the k -th column of A . The pricing problem asks whether there exists a column k such that $yA_k > g$, which is equivalent to finding a k such that reduced cost

$$\rho_k = g_k - yA_k < 0. \tag{B.3}$$

Observe that LP relaxation of the master problem (2.6) can be written as

$$\begin{aligned} \min \quad & [1 \quad d] \begin{bmatrix} \zeta \\ w \end{bmatrix} \\ \text{s.t.} \quad & \begin{bmatrix} I & A \\ 0 & -c \end{bmatrix} \begin{bmatrix} \zeta \\ w \end{bmatrix} \geq \begin{bmatrix} \mathbf{1} \\ -C \end{bmatrix}, \\ & \zeta, w \geq 0, \end{aligned} \tag{B.4}$$

where $\mathbf{1}$ is the all-ones column vector,

Say something about the upperbounds on variables not being needed. Can they be ignored? How to argue this?

$$\begin{aligned} A_{ik} &= \mathbb{I}(k \in \mathcal{K}_i), & i \in \mathcal{P}, k \in \mathcal{K}, \\ d_k &= \sum_{i \in \mathcal{N}} \mathbb{I}(k \in \mathcal{K}_i), & k \in \mathcal{K}. \end{aligned}$$

and c is the row vector holding the complexity of each rule. The dual is described by

$$\begin{aligned} \mathbf{max} \quad & [\mu \quad \lambda] \begin{bmatrix} \mathbf{1} \\ -C \end{bmatrix} \\ \mathbf{s.t.} \quad & [\mu \quad \lambda] \begin{bmatrix} I & A \\ 0 & -c \end{bmatrix} \leq [\mathbf{1} \quad d], \\ & \mu, \lambda \geq 0, \end{aligned} \tag{B.5}$$

Therefore, the reduced cost is given by

$$\begin{aligned} \rho_k &= d_k - (\mu A_k - \lambda c_k) \\ &= \sum_{i \in \mathcal{N}} \mathbb{I}(k \in \mathcal{K}_i) - \sum_{i \in \mathcal{P}} \mu_i \mathbb{I}(k \in \mathcal{K}_i) + \lambda c_k. \end{aligned} \tag{B.6}$$

B.2 Additive ramp loss model

The derivation of the reduced cost for the additive ramp loss model (4.1) is analogous to the derivation for the boolean model. Consider (B.4), only now A is replaced by Γ , where

$$\Gamma_{ik} = \gamma_{ik}, \quad i \in \mathcal{P}, k \in \mathcal{K},$$

the $\mathbf{1}$ -vector on the right-hand-side is replaced by a $\mathbf{2}$ -vector, and we choose

$$d_k = \sum_{i \in \mathcal{N}} \gamma_{ik}, \quad k \in \mathcal{K},$$

This formulation corresponds precisely to the additive ramp loss model (4.1). It follows that

$$\rho_k = \sum_{i \in \mathcal{N}} \gamma_{ik} - \sum_{i \in \mathcal{P}} \mu_i \gamma_{ik} + \lambda c_k. \tag{B.7}$$

Revise this section

B.3 Non-additive ramp loss model

The non-additive ramp-loss model formulation (4.15) can be written as

$$\begin{aligned} \mathbf{min} \quad & [\mathbf{1} \quad d] \begin{bmatrix} \zeta \\ w \end{bmatrix} \\ \mathbf{s.t.} \quad & \begin{bmatrix} \hat{I} & \hat{H} \\ 0 & -c \end{bmatrix} \begin{bmatrix} \zeta \\ w \end{bmatrix} \geq \begin{bmatrix} \mathbf{2} + \eta \\ -C \end{bmatrix}, \\ & \zeta, w \geq 0, \end{aligned} \tag{B.8}$$

where $\hat{I} \in \mathbb{R}^{(\mathcal{P} \times \mathcal{K}) \times \mathcal{P}}$, $\hat{H} \in \mathbb{R}^{(\mathcal{P} \times \mathcal{K}) \times \mathcal{K}}$ such that

$$\begin{aligned}\hat{I}_{[ik']i'} &= \mathbb{I}(i = i') & i, i' \in \mathcal{P}, \quad k' \in \mathcal{K}, \\ \hat{H}_{[ik']k} &= \eta_{ik'} - \eta_{ik}, & i \in \mathcal{P}, \quad k', k \in \mathcal{K},\end{aligned}$$

and c, d are the same as before. The dual then consists of variables $\mu \in \mathbb{R}_+^{\mathcal{P} \times \mathcal{K}}$ and $\lambda \in \mathbb{R}_+$. The reduced cost of a rule k is given by

$$\begin{aligned}\rho_k &= d_k - (\mu \hat{H}_k - \lambda c_k) \\ &= \sum_{i \in \mathcal{N}} \gamma_{ik} - \sum_{\substack{i \in \mathcal{P}, k' \in \mathcal{K} \\ \eta_{ik} < \eta_{ik'}}} \mu_{ik'} (\eta_{ik'} - \eta_{ik}) + \lambda c_k. \\ &= \sum_{i \in \mathcal{N}} \gamma_{ik} - \sum_{\substack{i \in \mathcal{P}, k' \in \mathcal{K} \\ \eta_{ik} < \eta_{ik'}}} \mu_{ik'} \eta_{ik'} + \sum_{\substack{i \in \mathcal{P}, k' \in \mathcal{K} \\ \eta_{ik} < \eta_{ik'}}} \mu_{ik'} \eta_{ik} + \lambda c_k \\ &= \sum_{i \in \mathcal{N}} \gamma_{ik} - \sum_{\substack{i \in \mathcal{P}, k' \in \mathcal{K} \\ \eta_{ik} < \eta_{ik'}}} \mu_{ik'} \eta_{ik'} + \sum_{i \in \mathcal{P}} \eta_{ik} \sum_{\substack{k' \in \mathcal{K} \\ \eta_{ik} < \eta_{ik'}}} \mu_{ik'} + \lambda c_k\end{aligned} \tag{B.9}$$

The chosen rule influences what elements are included in the sum. This makes translating the reduced cost to a MIP non-trivial and complicated.

The formulation (4.16) may be written as

$$\begin{aligned}\mathbf{min} \quad & \begin{bmatrix} 1 & d & 0 \end{bmatrix} \begin{bmatrix} \zeta \\ w \\ u \end{bmatrix} \\ \mathbf{s.t.} \quad & \begin{bmatrix} I & 0 & -\hat{H} \\ 0 & 0 & \hat{I}^u \\ 0 & 0 & -\hat{I}^u \\ 0 & \hat{I}^w & -I \\ 0 & -c & 0 \end{bmatrix} \begin{bmatrix} \zeta \\ w \\ u \end{bmatrix} \geq \begin{bmatrix} 2 \\ 1 \\ -1 \\ 0 \\ -C \end{bmatrix}, \\ & \zeta, w, u \geq 0,\end{aligned} \tag{B.10}$$

where $\hat{H}, \hat{I}^u \in \mathbb{R}^{\mathcal{P} \times (\mathcal{P} \times \mathcal{K})}$, $\hat{I}^w \in \mathbb{R}^{(\mathcal{P} \times \mathcal{K}) \times \mathcal{K}}$, given by

$$\begin{aligned}\hat{H}_{i[i',k]} &= \eta_{ik} \mathbb{I}(i = i'), & i \in \mathcal{P}, \quad [i', k] \in \mathcal{P} \times \mathcal{K}, \\ \hat{I}_{i[i',k]}^u &= \mathbb{I}(i = i'), & i \in \mathcal{P}, \quad [i', k] \in \mathcal{P} \times \mathcal{K}, \\ \hat{I}_{[ik']k}^w &= \mathbb{I}(k = k'), & [i, k'] \in \mathcal{P} \times \mathcal{K}, \quad k \in \mathcal{K},\end{aligned}$$

Again say something about the upperbounds on variables not being relevant... Is it true and how to argue?

The dual then consists of variables

$$\begin{aligned}\mu &\in \mathbb{R}_+^{\mathcal{P}}, \\ \nu_1 &\in \mathbb{R}_+^{\mathcal{P}}, \\ \nu_2 &\in \mathbb{R}_+^{\mathcal{P}}, \\ \theta &\in \mathbb{R}_+^{\mathcal{P} \times \mathcal{K}}, \\ \lambda &\in \mathbb{R}_+.\end{aligned}$$

The reduced cost of a rule k is given by

$$\begin{aligned}\rho_k &= d_k - \theta \hat{I}_k^w + \lambda c_k \\ &= \sum_{i \in \mathcal{N}} \gamma_{ik} - \sum_{i \in \mathcal{P}} \theta_{ik} + \lambda c_k.\end{aligned}\tag{B.11}$$

Appendix C

Cutting planes

C.1 Mixed Vertex Packing Problem

The Mixed Vertex Packing Problem, as described in [4], is defined as

$$\max \{cx + dy : (x, y) \in \text{MVP}\},$$

where

$$\text{MVP} = \{x \in \{0, 1\}^V, y \in \mathbb{R}^I : x_v + x_w \leq 1, \quad (v, w) \in E \quad (\text{C.1})$$

$$a_{vi}x_i + y_i \leq u_i, \quad (v, i) \in F \quad (\text{C.2})$$

$$0 \leq y_i \leq u_i \quad i \in I\}, \quad (\text{C.3})$$

is the Mixed Vertex Packing (MVP) polytope. Define

$$M(i) = \{v \in V : (v, i) \in F\}.$$

Proposition C.1.1 ([4]). for $i \in I$, let $T = \{v_1, v_2, \dots, v_t\} \subseteq M(i)$ such that $a_{v_{j-1}k} < a_{v_jk}$ for $j = 2, \dots, t$. Then the *star inequality*

$$\sum_{i \in T} \bar{a}_{vi}x_v + y_i \leq u_i, \quad (\text{C.4})$$

with

$$\begin{aligned} \bar{a}_{v_1k} &= a_{v_1k}, \\ \bar{a}_{v_jk} &= a_{v_jk} - a_{v_{j-1}k}, \quad j = 2, 3, \dots, t, \end{aligned}$$

is valid for $\text{conv}(\text{MVP})$.

If $E = \emptyset$, then $\text{conv}(\text{MVP})$ turns out to have a relatively simple description.

Theorem C.1.1 ([4]). If $E = \emptyset$, then inequalities (C.4) together with inequalities

$$\begin{aligned} 0 \leq x_v \leq 1, & & v \in V, & & \text{(C.5)} \\ 0 \leq y_i \leq u_i, & & i \in I, & & \text{(C.6)} \end{aligned}$$

are sufficient to describe $\text{conv}(\text{MVP})$.

In the MVP polytope, we recognize (4.11) with only the constraint (4.11b). Precisely, choose

$$\begin{aligned} V &= \mathcal{L}, \\ \mathbf{x} &= \mathbf{z}, \\ I &= \mathcal{I}, \\ \mathbf{y} &= \boldsymbol{\gamma}, \\ u_i &= 2, & i \in \mathcal{I} \\ a_{li} &= 2 - \gamma_{il}, & i \in \mathcal{I}, l \in \mathcal{L}. \end{aligned}$$

The star inequalities (C.4) for this instance yield the cutting planes (4.12).

revise this section

C.2 Extended Polymatroid Inequalities

Let

$$\gamma_1 \leq \gamma_2 \leq \cdots \leq \gamma_n$$

be given.

Definition C.2.1. A function $g : 2^N \rightarrow \mathbb{R}$ is *submodular* if for all $A \subseteq B \subseteq N$ and $j \notin B$,

$$\rho_j(B) \leq \rho_j(A), \quad \text{(C.7)}$$

where $\rho_j(I) := g(I \cup \{j\}) - g(I)$ is the marginal gain of j with respect to I . A function is *supermodular* if and only $-g$ is submodular. For a supermodular function, (C.7) is reversed.

Lemma C.2.1. The function f is submodular.

Proof. Let $A \subseteq B \subseteq [n]$ and let $j \in [n] \setminus B$. Since $A \subseteq B$, we have that

$$f(B) \leq f(A).$$

It holds that

$$\rho_j(A) = \begin{cases} \gamma_j - f(A) & \text{if } \gamma_j < f(A), \\ 0 & \text{otherwise.} \end{cases}$$

The formula for $\rho_j(B)$ is analogous. To prove the statement, we condition on $\rho_j(B)$. If $\rho_j(B) = \gamma_j - f(A)$, then

$$\rho_j(B) = \gamma_j - f(B) \geq \gamma_j - f(A) = \rho_j(A).$$

If $\rho_j(B) = 0$, then $\rho_j(B) \geq \rho_j(A)$, as $\rho_j(A)$ is non-positive. Therefore

$$\rho_j(B) \geq \rho_j(A).$$

This implies f is supermodular. $\square$.

Consider now Q , which is the hypograph of f on the boolean hypercube. Apply the substitution

$$\omega = -\gamma + f(0), \quad g = -f + f(0).$$

Then

$$Q = \{(z, \omega) \in \{0, 1\}^n \times \mathbb{R} : \omega \geq g(z)\}, \quad (\text{C.8})$$

which is the epigraph of g over the boolean hypercube, with g submodular and $g(0) = 0$. This object has been widely studied.

Definition C.2.2 (Extended polymatroid inequality). Given any permutation δ of $[n]$, the corresponding extended polymatroid inequality (EPI) is given by

$$\omega \geq \sum_{i=1}^n \rho_{\delta(i)}(X^{\delta, i-1}) x_{\delta(i)},$$

where $X^{\delta, i} = \{\delta(1), \dots, \delta(i)\}$ for any $i \in [n]$, $X^{\delta, 0} = \emptyset$, and ρ is the marginal gain function with respect to g . Alternatively, one may define

$$x = (x_{\delta(i)})_{i=1}^n, \quad \pi^\delta = \left(\rho_{\delta(i)}(X^{\delta, i-1}) \right)_{i=1}^n$$

and describe the inequality as

$$\omega \geq \pi^\delta x^\delta,$$

where the right hand side is an inner product.

Theorem C.2.1 ([15, 3, 20]).

$$\text{conv}(Q) = \left\{ (z, \omega) \in [0, 1]^n \times \mathbb{R} : \omega \geq \pi^\delta x^\delta, \delta \text{ a permutation of } [n] \right\}. \quad (\text{C.9})$$

Furthermore, the inequality is facet-defining if and only if π^δ is an extreme point of

$$\left\{ \pi \in \mathbb{R}^n : \sum_{i \in I} \pi_i \leq g(I) \text{ for all } I \subseteq [n] \right\}.$$

Proposition C.2.1. $\text{conv}(Q) = P$.

Proof. We will prove this by showing that the inequalities in (C.9) are exactly those in $\bar{P}$. First observe that

$$\begin{aligned} \bar{P} = \{(\mathbf{z}, \gamma) \in [0, 1]^n \times \mathbb{R} \mid \gamma \leq \gamma_n - \sum_{j=1}^k (\gamma_{i_{j+1}} - \gamma_{i_j}) z_{i_j}, \\ i_1 < i_2 < \dots < i_k \in [m-1], \text{ setting } i_{k+1} = m\} \end{aligned} \quad (\text{C.10})$$

Let δ be a permutation of $[n]$. It holds that

$$\rho_{\delta(i)}(X^{\delta, i-1}) = \begin{cases} f(X^{\delta, i-1}) - \gamma_{\delta(j)} & \text{if } \gamma_{\delta(i)} < f(X^{\delta, i-1}), \\ 0 & \text{otherwise.} \end{cases}$$

Observe that

$$\gamma_{\delta(i)} < f(X^{\delta, i-1}) \iff \delta(i) < \delta(j) \text{ for all } j < i.$$

Let $(i_k)_{k=1}^m$ be the subsequence of indices of δ such that

$$\delta(i_k) < \delta(j) \text{ for all } j < i_k,$$

i.e. the positions of successive minima of δ . These satisfy $i_1 < i_2 < \dots < i_m$. Setting i_0 such that $\delta(i_0) = n$ (i.e. the implicit starting value), we have

$$f(X^{\delta, i_k-1}) = \gamma_{\delta(i_{k-1})},$$

and hence the marginal contribution at each new-minimum position is

$$\rho_{\delta(i_k)}(X^{\delta, i_k-1}) = \gamma_{\delta(i_{k-1})} - \gamma_{\delta(i_k)}.$$

It follows that

$$\omega \geq \sum_{k=1}^m (\gamma_{\delta(i_{k-1})} - \gamma_{\delta(i_k)}) z_{\delta(i_k)}.$$

Writing $j_k = \delta(i_k)$, we have $j_0 = n > j_1 > \dots > j_m$, so reversing via $\hat{j}_k = j_{m+1-k}$ gives $\hat{j}_1 < \dots < \hat{j}_m < n$, and setting $\hat{j}_{m+1} = n$:

$$\gamma \leq \gamma_n - \sum_{k=1}^m (\gamma_{\hat{j}_{k+1}} - \gamma_{\hat{j}_k}) z_{\hat{j}_k},$$

which is exactly an inequality of $\bar{P}$. By arguing the other way, there is a bijection between inequalities in $\text{conv}(Q)$ and $\bar{P}$. $\square$

Appendix D

Results of experiments

D.1 Boolean model integer gap

The integer gap is defined as

$$\frac{\text{Optimal value of the IP} - \text{Optimal value of the LP}}{\text{Optimal value of the LP}}.$$

The optimal value of the LP is a lower bound for the optimal value of an IP. When pricing completed, which is true for green cells that do not say “Limit”, the LP relaxation value observed is the true optimal LP relaxation value. Hence, no integer solution with an objective value better than it exists. Since the gap is small for these cases, we conclude that there do not exist meaningfully better rulesets than the ones we found. It also implies branch-and-price is not meaningfully better than price-then-branch in this case. For the cases where pricing did not converge, this conclusion cannot be made.

Table D.1 shows that pricing finished for all instances, and this argument holds. Table D.2 shows that pricing only finished in some cases.

Table D.1: Integer gap for the boolean model on the full Broad dataset for various $n_{\text{quantiles}}$ and Q_y , only lower-bounds

$n_{\text{quantiles}}$ Q_y	1	3	5	7	9
0.5	0.000	0.000	0.004	0.010	0.003
0.6	0.000	0.012	0.000	0.001	0.000
0.7	0.000	0.000	0.000	0.002	0.000
0.8	0.000	0.005	0.000	0.002	0.000
0.9	0.000	0.000	0.000	0.000	0.000

Table D.2: Integer gap for the boolean model on the full Broad dataset for various $n_{\text{quantiles}}$ and Q_y , both lower-bounds and upper-bounds

$n_{\text{quantiles}}$ Q_y	1	3	5	7	9
0.5	0.028	Limit	Limit	Limit	Limit
0.6	0.015	Limit	Limit	Limit	Limit
0.7	0.003	0.009	Limit	Limit	Limit
0.8	0.000	0.009	Limit	Limit	Limit
0.9	0.000	0.000	0.011	0.000	Limit

D.2 Ramp loss model generalization

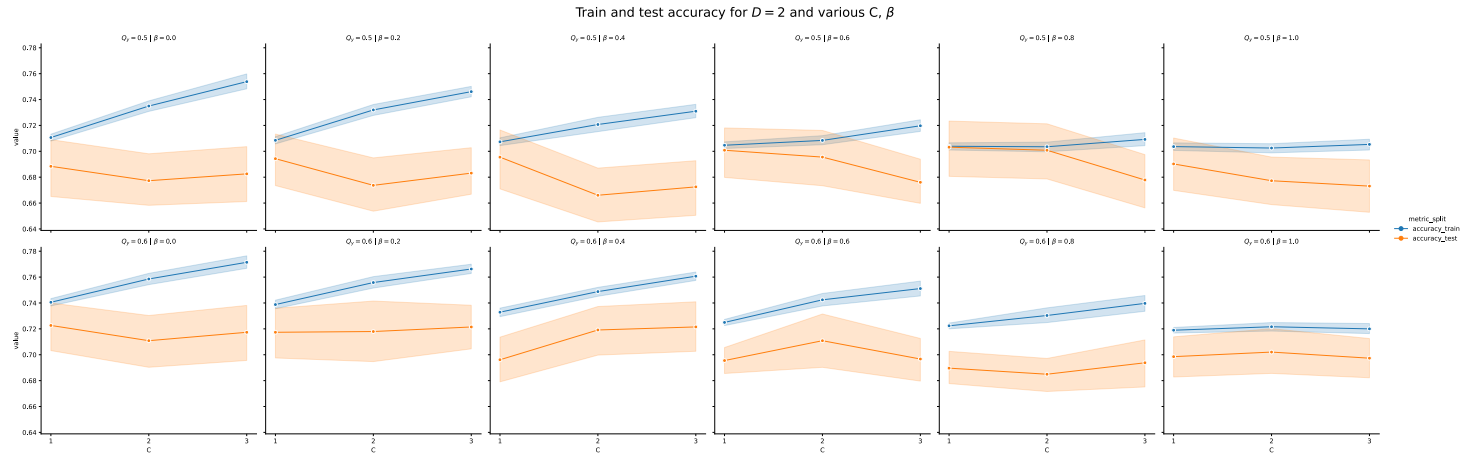


Figure D.1: Confidence intervals for train and test accuracies for the ramp loss model, $C = 1, 2, 3$, $D = 2$, $\beta = 0, 0.2, 0.4, 0.6, 0.8, 1$.

D.3 Gene expression as input

Table D.3: Results for the full Broad dataset, using gene expression as input, for $C = 1$ and small rules ($D = 1, 2, 3$).

Q_y	C	D	Acc.	Prec.	Rec.	Ruleset
0.5	1	1	0.71	0.68	0.81	$\{FOSL1 \geq 30.61 Q(0.40)\}$
		2	0.73	0.75	0.70	$\{FOSL1 \geq 30.61 Q(0.40) \wedge$ $STAT4 \geq 0.17 Q(0.31)\}$
		3	0.74	0.76	0.70	$\{FOSL1 \geq 30.61 Q(0.40) \wedge$ $STAT4 \geq 0.17 Q(0.31) \wedge$ $NFATC1 \leq 9.69 Q(0.90)\}$
0.6	1	1	0.74	0.68	0.68	$\{FOSL1 \geq 109.35 Q(0.60)\}$
		2	0.76	0.75	0.60	$\{FOSL1 \geq 109.35 Q(0.60) \wedge$ $NFATC1 \leq 3.92 Q(0.70)\}$
		3	0.76	0.75	0.60	$\{AR \geq 0.00 Q(0.28) \wedge$ $FOSL1 \geq 109.35 Q(0.60) \wedge$ $NFATC1 \leq 3.92 Q(0.70)\}$

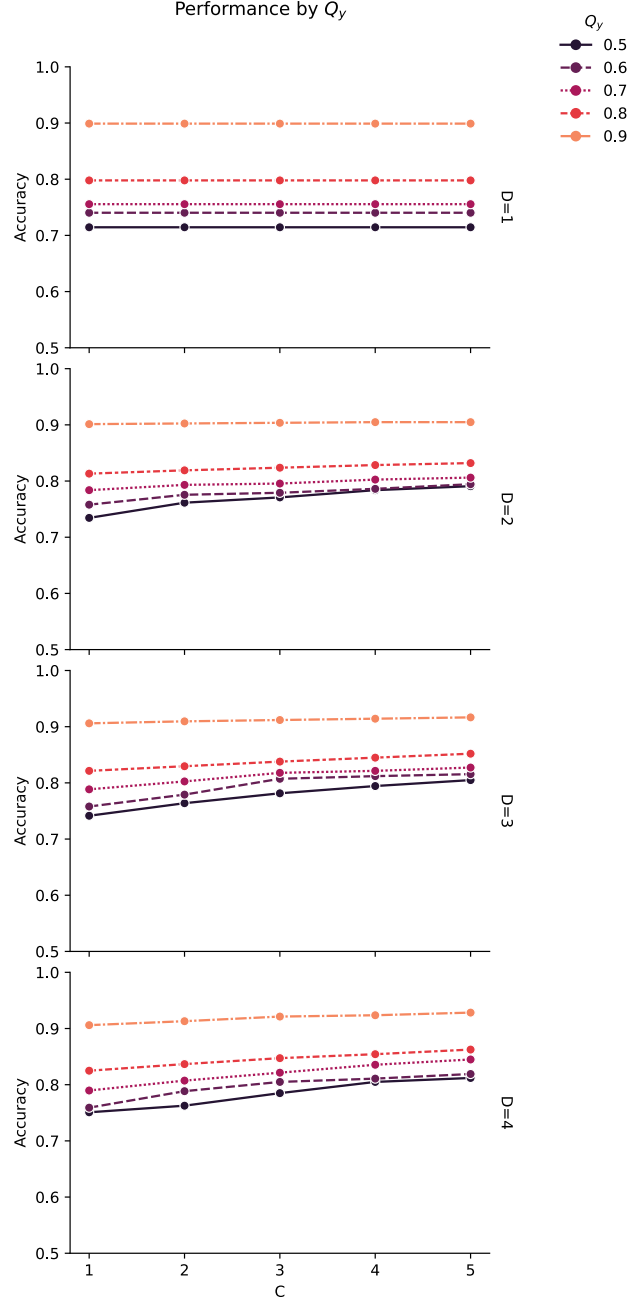


Figure D.2: Accuracy of results for the full Broad dataset, using gene expression as input, for $C = 1, 2, 3, 4, 5$, $D = 1, 2, 3, 4$ and $Q_y = 0.5, 0.6, 0.7, 0.8, 0.9$.

D.4 Transcription Factor Exclusion

Table D.5: TF exclusion table as described in Section 5.3.
 Case $Q_y = 0.5$ (top) and $Q_y = 0.6$ (bottom), $D = 1$ (left) and $D = 2$ (right).

Depth	Baseline: 0.69	Baseline: 0.71							
1	-STAT2: 0.69	-FOS: 0.71				-STAT2: 0.69			
2	-RELA: 0.66	-RELA: 0.70		-STAT2: 0.69		-RELA: 0.68		-SOX2: 0.69	
3	-FOSL1: 0.66	-NFKB1: 0.70	-STAT2: 0.68	-RELA: 0.68	-SOX2: 0.69	-FOSL1: 0.67	-NFKB1: 0.68	-FOS: 0.69	-RELA: 0.68

Depth	Baseline: 0.70	Baseline: 0.74							
1	-FOSL1: 0.70	-FOSL1: 0.73				-STAT2: 0.73			
2	-STAT2: 0.69	-RELA: 0.72		-STAT2: 0.71		-FOSL1: 0.71		-RELA: 0.73	
3	-RELA: 0.69	-FOS: 0.72	-STAT2: 0.70	-RELA: 0.70	-SOX2: 0.71	-RELA: 0.70	-SOX2: 0.71	-FOS: 0.72	-FOSL1: 0.70